

Playable Worlds Lab - Unified Source of Truth v4.1

Table of Contents

Playable Worlds Lab - Unified Source of Truth v4.1

Teen/Adult Edition: Product Strategy, Architecture, Implementation Plan, and AI Tool Operator Guide

Status: Active source of truth for pre-code implementation

Date: 2026-05-26

Core mantra: AI proposes. Validators check. The game engine executes.

This document replaces the active working use of the Product Build Report v3, AI Coding Tool Execution Playbook, and Preflight Revision Pack v4. Older PDFs are archived references only. This v4.1 document is the one human operator and AI coding tools should follow before starting W1-S1.

0. Project Summary: What We Are Building and Why

Playable Worlds Lab is a platform for creating, playing, evolving, testing, sharing, and remixing AI-directed game worlds. The first version is intentionally text-first. We are not trying to build a large 3D metaverse on day one. We are first proving the world engine underneath it.

The project starts with one proof world: **Stonepass Valley**. A player reaches a bridge blocked by an ogre. The player can fight, trick, feed, talk to, or sneak around the ogre. Each choice changes remembered world state. One path causes a landslide, exposes a temporary cave, sends the player through a short instance, and awakens a dragon. That small chain proves the core systems before bigger generation, sharing, or visual layers are added.

Question	Answer
What are we building?	A schema-first, text-first AI-directed world engine where player choices change remembered world state.
Why text-first?	Text lets us prove logic, memory, validation, replayability, and AI direction before investing in visuals.
Why Stonepass first?	It is the smallest world that proves choices, consequences, temporary instances, ledger memory, AI suggestions, and validation.
What does AI do?	AI suggests next beats, NPC reactions, recaps,

Question	Answer
What does AI not do?	temporary instances, critiques, and generated drafts. AI does not directly mutate permanent world truth. The deterministic game engine executes changes.
What keeps the world reliable?	Zod schemas, complete WorldDefinition validation, cross-file graph validation, AIResult wrappers, fallbacks, health checks, and AI Playtester reports.
What is the long-term bet?	AI game worlds need structure, memory, validation, playtesting, and versioning before 2D/3D layers can scale.

The product in one sentence

Build Stonepass Valley as a browser-playable, text-first AI-directed world where player choices change remembered world state, AI suggests validated next steps, a temporary cave can awaken a dragon, and the whole world can later be saved, tested, shared, forked, and remixed safely.

The deeper product bet

The future of AI games is not just generating pretty scenes from prompts. The stronger bet is that a world should be generated as structured, validated, inspectable, replayable game logic first. Visual presentation can come later as an output layer.

Idea

- > WorldDefinition
- > Story beats
- > Choices
- > Consequences
- > Ledger memory
- > AI suggestions
- > Validation
- > Playtesting
- > Save/share/fork/remix
- > Later visual runtime

1. Replay Variation and Market Differentiation Model

Existing text and interactive-fiction games already use choices, branches, variables, routes, classes, stats, and alternate endings. Playable Worlds Lab should respect that pattern. The difference is that this project turns those ideas into a validated world engine with AI direction, memory, creator inspection, playtesting, sharing, forking, remixing, and later visual layers.

Existing text-game pattern	Playable Worlds Lab version
Authored branching story	Authored branches plus validated AI-suggested branches.
Class/background choices	Lightweight RunProfile/archetype variation that changes flavor, choices, and available routes.
Variables and conditionals	World Ledger flags, goals, NPC attitudes, DebugEvents, and session state.
Slightly different playthroughs	Reasoned variation from archetype, route, consequence flags, seeds, AI Director suggestions, and temporary instances.
Writer manually tests paths	Cross-file validator, deterministic path runner, health score, and AI Playtester.
Static release content	Versioned worlds that can be shared, forked, remixed, patched, and later visually rendered.
Hidden internal logic	Creator/debug panels expose why a state changed or why AI suggested something.

Replay variation layers

Layer	Name	Example	When to build
1	Starting archetype	mage, warrior, rogue, diplomat	Lightweight Phase 1/2 support only
2	Starting route	bridge, town, castle, forest	Phase 1/2 as flavor or small route changes
3	Consequence branches	ogre defeated, allied, tricked, avoided	Phase 1
4	Conditional text	mage sees runes; warrior sees battle marks	Phase 1/2
5	NPC memory	elder trusts or fears player	Phase 2
6	Temporary instance variants	puzzle cave vs creature cave	Phase 3
7	AI Director variation	next beat adapts to current world state	Phase 2/3
8	Remix/fork variation	alternate timeline/version	Phase 6
9	Generated world variation	prompt creates a new valid world	Phase 5+

Replay variation guardrails

- Every variation must be explainable by profile, route, seed, flag, NPC attitude, validated AI suggestion, or approved remix.
- Do not use random AI improvisation as a substitute for world logic.
- Do not build full RPG stats, inventory, economy, skill trees, or combat math in the MVP.
- MVP replay variation may use lightweight archetype, route, consequence flags, conditional text, and NPC attitude only.

Optional future lightweight contract:

```
type PlayerArchetype = "warrior" | "mage" | "rogue" | "diplomat";
```

```
type RunProfile = {  
  archetype: PlayerArchetype;  
  startingRoute?: "bridge" | "town" | "castle" | "forest";  
  personalityTags?: string[];  
};
```

This belongs after the core Stonepass foundation is stable. It is a replay-flavor layer, not a full character-builder system.

2. How to Use This Document

Rule	Human operator	AI coding tool
One step at a time	Approve exactly one current step.	Implement only that step and stop.
This document wins	Use v4.1 as the active source.	Ignore older conflicting PDFs unless human explicitly quotes them.
Scope control	Reject opportunistic extra features.	List allowed and blocked scope before coding.
External setup	Create repos, accounts, keys, and payment/provider setup.	Create placeholders and stop when external setup is needed.
Validation first	Require tests and acceptance evidence.	Add schemas/validators/tests before calling a feature done.
Completion report	Review evidence before next step.	Report files changed, tests run, result, blockers, next safe step.

Source priority order

Priority	Source
1	Playable Worlds Lab - Unified Source of Truth v4.1

Priority	Source
2	Current human-approved Cursor step prompt
3	Existing repository code
4	Archived Product Build Report v3 / AI Playbook / Preflight Pack, only when used as historical context
5	Cursor or AI assistant suggestions

If two instructions conflict, stop. Do not guess. Report the conflict and ask the human owner which instruction wins. The step prompt can narrow scope, but it cannot expand beyond this source of truth or bypass a blocked phase.

3. Core Operating Principles

Principle	Meaning
AI proposes	AI may suggest beats, NPC reactions, recaps, temporary instances, critiques, or generated drafts.
Validators check	All AI/generated/manual content must pass schemas and deterministic validators before use.
Engine executes	Only deterministic runtime code mutates WorldSession, WorldLedger, and permanent world truth.
Text first	Prove the world engine before 2D/3D presentation.
Stonepass first	Stonepass Valley is the proof path for every core system.
Teen/adult only	No lower-age mode exists in the MVP. Do not add another audience mode without a new source-of-truth revision.
One step, one report	Every implementation step ends with test evidence and a stop.

4. Audience and Content Safety Contract

Playable Worlds Lab v4.1 is a teen/adult project. There is no lower-age public mode in the MVP. Official packs may be lighter, cozy, or educational in tone, but they still use the teen/adult safety contract.

type SafetyMode = "teen" | "adult";

Mode	Allowed	Not allowed
teen	Adventure-safe tone, fantasy	Explicit sexual content,

Mode	Allowed	Not allowed
adult	danger, suspense, non-graphic combat, mystery, limited mild romance references.	graphic gore, hate content, self-harm instruction, real-world extremist praise, exploitative content.
	Darker stakes, stronger danger, mature themes, sparing stronger language when tone supports it.	Explicit sexual content, graphic gore, hate content, self-harm instruction, real-world extremist praise, exploitative content.
• Public discovery remains blocked until quality and moderation gates exist. • AI-generated content must be validated against safetyMode before it is saved, shared, or used as official world data. • NPCs can be hostile, afraid, strange, funny, heroic, or mysterious, but must not encourage real-world harm, hate, exploitation, or illegal instruction. • Blocked until safety and quality gates: public UGC, public discovery, creator monetization, social chat, voice, and public remix libraries.		

5. Product Definition

Playable Worlds Lab turns game ideas into living, shareable, AI-directed playable worlds. Users can create a world, play through choices, trigger consequences, generate next story beats, fork alternate timelines, and eventually connect worlds through portals and player identity.

What it is	Meaning
Game concept compiler	Transforms vague ideas into structured, playable world blueprints.
World-state engine	Tracks flags, choices, consequences, locations, NPC attitudes, temporary instances, and history.
AI Director runtime	Chooses or suggests next meaningful quest, hint, pacing, NPC reaction, recap, or temporary instance.
Creator tool	Lets users generate, patch, critique, playtest, share, fork, and remix worlds.
Future network of worlds	Eventually supports player profiles, world passports, artifacts, portals, and official/community worlds.
What it is not at first	Reason
Not a Roblox clone	We are not starting with public creation chaos or multiplayer platform complexity.
Not a public UGC discovery platform at	Quality and safety gates must exist first.

What it is not at first	Reason
launch	
Not a VR-only metaverse	The world engine comes before visual spectacle.
Not a AAA 3D open-world generator	3D is a later output layer, not the foundation.
Not an AI agent with full control over truth	AI suggestions must pass validators and deterministic engine execution.

6. Stonepass Valley MVP Scope

Stonepass Valley is the first implementation target: the smallest world that proves the magic.

Player reaches an ogre-blocked bridge

- > Player chooses fight, trick, feed, talk, or sneak
- > Each choice creates a different consequence
- > Main proof path causes landslide
- > Hidden cave opens
- > Player enters temporary cave
- > Cave has room/encounter/puzzle/relic moment
- > Cave collapses/vanishes/resolves
- > Dragon awakens
- > New story goal appears
- > World can later be saved/shared/forked/remixed

Stonepass proof path ownership

Phase	Owns
Phase 1	Deterministic text choices, consequence application, ledger updates, debug events, and next-goal selection.
Phase 3	landslide -> temporary cave -> dragon awakening chain.
Phase 5+	Prompt-to-world generation only after manual Stonepass runtime support is proven.

MVP must include

- Text-based playable world
- World DNA
- WorldDefinition object
- Story Beat Graph
- NPC profiles and tone rules
- World Memory Ledger
- WorldSession save-state object
- Consequence Engine

- DebugEvent log
- AIResult wrapper for validated AI calls
- AI Director v1
- Temporary Instance Generator v1
- NPC dialogue generation
- World state save/load
- Debug/Director reasoning panel
- Shareable world state
- Forkable world state
- World Health Score v1
- Cross-file world validator
- AI Playtester v1

MVP must not include yet

- 2D map editor
- 3D rendering
- Real-time multiplayer
- Public world marketplace
- Creator monetization
- Voice chat
- Open public UGC discovery
- Complex avatar system
- Full economy
- PvP
- VR

7. Core Product Pillars

Pillar	What it does	MVP proof
World DNA	Defines genre, tone, loop, session length, safety mode, consequence intensity, creativity.	Stonepass world config.
WorldDefinition	One complete validated world object.	Stonepass validates as a whole.
Story Beat Graph	Branching moments with triggers, choices, consequences, and next hooks.	Ogre bridge choices.
Consequence Engine	Applies deterministic world changes from player actions.	fight/feed/talk/trick/sneak consequences.
World Ledger	Records flags, events, locations, visible changes,	World remembers the ogre choice.

Pillar	What it does	MVP proof
WorldSession	and goals. Tracks current beat, turn, active instance, choice history, ledger, debug events.	Save/resume and runtime state.
DebugEvent Log	Explains why something changed or why validation/fallback happened.	Debug panel.
AI Gateway / AIResult	Routes AI calls through schemas, wrappers, fallback, and traceability.	Director/NPC fallback.
Temporary Instances	Short caves, ruins, dreams, trials, dungeons that resolve back to world.	Hidden cave awakens dragon.
AI Playtester	Finds broken paths, missing goals, bad flags, weak consequences.	Stonepass passes; broken fixture fails.
Share/Fork/Remix	Portable, replayable, branchable worlds without overwriting originals.	Later MVP gates.

8. Tech Stack and Repository Structure

Area	Choice	Reason
App framework	Next.js + React + TypeScript	Fast browser MVP, API routes, easy UI iteration.
UI styling	Tailwind CSS	Fast layouts and simple components.
Validation	Zod	Runtime schema validation for world and AI output.
Testing	Vitest, then Playwright	Unit tests first; browser flow tests later.
AI provider layer	Custom provider interface	Keeps OpenAI/Anthropic/Gemini/Ollama swappable.
Database later	Supabase Postgres	Quick MVP backend after local runtime is stable.
2D later	Phaser	Browser-based 2D output layer after text gate.

Package and runtime setup lock

- Use npm unless the human owner explicitly chooses another package manager before W1-S1.
- Add .nvmrc during W1-S1.
- Add package.json engines during W1-S1.
- Do not mix npm, pnpm, and yarn lockfiles.
- Add scripts for dev, build, test, typecheck, lint, and content validation before feature work expands.
- Cursor must report package manager and Node version in W1-S1 completion report.

Repository structure

```
apps/  
  web/  
    app/  
  components/  
  features/  
    world-create/  
    world-play/  
    world-debug/  
    world-share/  
  creator/  
docs/  
  source-priority.md  
  content-safety-rules.md  
  decision-log.md  
packages/  
  core/  
    world/  
  story/  
  ledger/  
  consequence/  
  runtime/  
  session/  
  debug/  
  validators/  
    validateWorldDefinition.ts  
  migrations/  
  health/  
  playtest/  
  schemas/  
ai/  
  gateway/  
  contracts/
```

```
providers/  
  fakeProvider.ts  
  openaiProvider.ts  
  anthropicProvider.ts  
  geminiProvider.ts  
  ollamaProvider.ts  
agents/  
  worldArchitectAgent.ts  
  directorAgent.ts  
  consequenceAgent.ts  
  instanceGeneratorAgent.ts  
  npcAgent.ts  
  playtesterAgent.ts  
  criticAgent.ts  
content/  
  templates/  
  worlds/  
  npcs/  
  creatures/  
  story-beats/  
tests/  
  unit/  
  integration/  
  fixtures/
```

9. Canonical Data Contracts

These contracts are canonical. Cursor must not invent alternate object shapes unless a new human-approved source-of-truth revision changes them.

```
type SafetyMode = "teen" | "adult";
```

```
type WorldDNA = {  
  genre: "fantasy" | "sci-fi" | "mystery" | "cozy" | "survival";  
  tone: "cozy" | "heroic" | "dark" | "funny" | "mysterious";  
  sessionLengthMinutes: 5 | 10 | 15 | 30 | 45;  
  coreLoop: string[];  
  consequenceIntensity: "light" | "medium" | "major";  
  aiCreativity: "conservative" | "balanced" | "wild";  
  safetyMode: SafetyMode;  
};
```

```
type PlayerChoice = {  
  id: string;
```

```
label: string;
description?: string;
requiredFlags?: string[];
blockedByFlags?: string[];
consequenceId: string;
};
```

```
type StoryBeat = {
  id: string;
  title: string;
  description: string;
  trigger: string;
  availableChoices: PlayerChoice[];
  possibleConsequences: string[];
  requiredFlags?: string[];
  blockedByFlags?: string[];
  isEnding?: boolean;
  isHidden?: boolean;
};
```

```
type WorldEvent = {
  id: string;
  type: "choice" | "consequence" | "flag" | "goal" | "instance" | "ai" | "system";
  summary: string;
  turnNumber: number;
  metadata?: Record<string, unknown>;
};
```

```
type WorldLedger = {
  activeFlags: string[];
  resolvedFlags: string[];
  worldEvents: WorldEvent[];
  discoveredLocations: string[];
  unlockedGoals: string[];
};
```

```
type Consequence = {
  id: string;
  summary: string;
  addFlags: string[];
  removeFlags: string[];
  visibleChanges: string[];
  unlockGoals: string[];
};
```

```
    npcUpdates?: string[];
    temporaryInstances?: string[];
};
```

```
type NPC = {
    id: string;
    name: string;
    role: string;
    description: string;
    attitude?: "friendly" | "neutral" | "hostile" | "afraid" | "curious";
    toneRules?: string[];
    knownFlags?: string[];
};
```

```
type TemporaryInstanceRoom = {
    id: string;
    title: string;
    description: string;
    interactions: string[];
    encounter?: string;
    puzzle?: string;
};
```

```
type TemporaryInstance = {
    id: string;
    title: string;
    type: "cave" | "ruin" | "trial" | "dream" | "dungeon";
    requiredEntryFlags: string[];
    entranceText: string;
    rooms: TemporaryInstanceRoom[];
    completionCondition: string;
    completionConsequenceId: string;
    cleanupBehavior: "vanish" | "collapse" | "seal" | "resolve";
    generationSeed?: string;
};
```

```
type WorldDefinition = {
    schemaVersion: string;
    id: string;
    title: string;
    summary: string;
    worldDNA: WorldDNA;
    startingBeatId: string;
};
```

```
storyBeats: StoryBeat[];
consequences: Consequence[];
npcs: NPC[];
temporaryInstances?: TemporaryInstance[];
generationSeed?: string;
};
```

```
type DebugEvent = {
  id: string;
  turnNumber: number;
  type: "choice_selected" | "consequence_applied" | "flags_changed" | "goal_unlocked" |
  "ai_suggestion" | "fallback_used" | "validation_failed" | "session_loaded" |
  "session_saved";
  summary: string;
  metadata?: Record<string, unknown>;
};
```

```
type WorldSession = {
  id: string;
  schemaVersion: string;
  worldId: string;
  worldVersionId: string;
  currentBeatId: string;
  ledger: WorldLedger;
  activeTemporaryInstanceId?: string;
  currentTemporaryRoomId?: string;
  turnNumber: number;
  choiceHistory: string[];
  debugEvents: DebugEvent[];
};
```

```
type DirectorDecision = {
  action: "select_next_beat" | "generate_consequence" | "generate_instance" |
  "generate_npc_reaction" | "summarize_world" | "suggest_session_wrapup";
  targetId: string;
  reason: string;
  confidence: number;
  safetyNotes?: string[];
};
```

```
type AIResult<T> = {
  ok: boolean;
  value?: T;
```

```

raw?: unknown;
provider: string;
fallbackUsed: boolean;
validationErrors?: string[];
latencyMs?: number;
generationSeed?: string;
};

```

10. Runtime Loop and AI Rules

Core runtime loop

WorldDefinition validates as a complete object

- > WorldSession loads current WorldDefinition + WorldLedger
- > Runtime selects current StoryBeat
- > Player chooses action
- > Choice resolver validates choice availability
- > Consequence Engine validates and applies changes
- > Ledger records event
- > DebugEvent records why the change happened
- > AI Gateway may request Director/NPC suggestion
- > AIResult wraps raw output, validation result, fallback status, and latency
- > Validators check AI output
- > UI shows next goal, world changes, ledger, and reasoning

AI implementation rule

Use a provider interface. Do not call a specific model directly from random game systems. DirectorAgent, NPCReactionAgent, ConsequenceAgent, WorldArchitectAgent, PlaytesterAgent, and CriticAgent should all call an AI Gateway. The runtime must work with FakeProvider for tests.

```

interface AIProvider {
  generateStructured<T>(input: AIRequest, schema: ZodSchema<T>): Promise<T>;
}

```

11. Validators and Quality Gates

Validation layers

Layer	Purpose	Required before
Zod object schemas	Validate object shape, required fields, enums, and safe defaults.	Any runtime or AI use.
Cross-file world validator	Validate complete world graph references, reachability, flags, and dead	Runtime load, generated worlds, share/fork/remix, official templates.

Layer	Purpose	Required before
	ends.	
AI output validator	Validate structured AI output and wrap in AIResult.	Any AI suggestion reaches UI/runtime.
Content safety validator	Respect teen/adult contract.	Generated content is saved/shared/used.
Health Score	Explain quality and structural risk.	Generated worlds and creator cockpit.
AI Playtester	Explore paths and report soft-locks/missing goals/weak consequences.	Broader sharing and creator workflows.

Cross-file world validator must reject

- startingBeatId does not point to an existing StoryBeat.
- PlayerChoice consequenceId does not point to an existing Consequence.
- StoryBeat possibleConsequences entry does not point to an existing Consequence.
- Consequence temporaryInstances entry does not point to an existing TemporaryInstance.
- TemporaryInstance completionConsequenceId does not point to an existing Consequence.
- Duplicate IDs exist across story beats, choices, consequences, NPCs, or temporary instances.
- Required or blocked flags are misspelled or never produced, unless explicitly marked external/system flags.
- Story beat is unreachable and not marked isHidden or isEnding.
- Non-ending beat has no available choices and no next hook.
- Generated world has no clear starting beat, goal, choice, and consequence.

Schema version and deterministic seed rules

- Every WorldDefinition must include schemaVersion.
- Every saved WorldSession must include schemaVersion.
- Starting schemaVersion is 0.1.0 unless changed before W1-S9.
- Future migrations live under packages/core/migrations.
- Generated worlds, temporary instances, fake-provider outputs, and AI debug records may include generationSeed.
- If generated output fails validation, log the seed in DebugEvent or AIResult so the bug can be reproduced.

12. Phase Roadmap Overview

Phase	Goal	Build	Acceptance gate
0. Foundation	Define product rules, contracts, validators, provider interface,	Schemas, WorldDefinition, WorldSession,	All schemas validate; broken examples fail; Stonepass validates

Phase	Goal	Build	Acceptance gate
	Stonepass data.	DebugEvent, AIResult, validator, FakeProvider, manual Stonepass JSON.	as one world; FakeProvider works.
1. Text Runtime	Make Stonepass playable through deterministic text choices.	World loader, session, story selector, choice resolver, consequence engine, ledger/debug, text play screen.	Player starts Stonepass, chooses fight/feed/sneak/talk/trick, applies consequence, updates ledger/debug, reaches next valid goal without AI.
2. AI Director v1	Add AI suggestions without AI owning truth.	AI Gateway, DirectorAgent, NPCReactionAgent, AIResult, validators, fallback, reasoning panel.	Valid suggestions pass; invalid output rejected; fallback fires; game remains playable if AI fails.
3. Temporary Instances	Generate/play short cave/ruin/trial spaces that return to main world.	Instance generator/loader, rooms, encounter, puzzle, completion, cleanup.	Ogre -> landslide -> cave -> dragon works; cave resolves and ledger/debug update.
4. Browser UI and Persistence	Make text product usable in browser with save/resume.	Play page, state panel, ledger panel, reasoning panel, local persistence or approved DB.	User can play/resume in browser; no terminal required.
5. Prompt-to-World	Generate basic valid playable text worlds from prompts.	WorldArchitectAgent , DNA/title/summary /starting beat, graph generation, preview, validator.	Prompt creates valid WorldDefinition with goal, choice, consequence, and playable text start.
6. Share/Fork/Remix	Make worlds portable and replayable without overwriting originals.	Share links, snapshot/fresh start, fork, remix, versions, patch notes.	Fork/remix do not modify original; invalid tokens fail gracefully.
7. AI Playtester	Prevent broken worlds from approval/sharing.	Path runner, missing goal/consequence checks, flag dependency checks,	Broken worlds flagged; valid Stonepass passes; report stored.

Phase	Goal	Build	Acceptance gate
8+. Later visual/platform layers	Advance only after earlier gates.	reports. 2D runtime, creator cockpit, passport, official packs, co-op, 2.5D/3D.	Acceptance criteria defined before implementation starts.

13. Week-by-Week Implementation Plan

Week	Phase	Theme	Main output	Human setup	Hard stop
1	Phase 0	Foundation + schemas	Repo skeleton, contracts, validators, Stonepass JSON, FakeProvider .	Repo/ folder/ package manager.	No runtime expansion beyond validation.
2	Phase 1	Text runtime basics	Loader, session, beat selector, choice resolver, first play screen.	Local app check.	No AI.
3	Phase 1	Consequence /ledger/ debug	Consequence Engine, flag rules, ledger/debug panels.	Review first play path.	No AI or generated content.
4	Phase 2	AI Gateway + Director	Gateway, DirectorAgent, NPCReaction Agent, fallback, reasoning panel.	Optional API key if real provider requested.	No prompt-to-world.
5	Phase 3	Temporary cave proof path	Cave entry, rooms, puzzle/encounter, cleanup, dragon awakening.	Review cave tone.	No prompt-to-world.

Week	Phase	Theme	Main output	Human setup	Hard stop
6	Hardening	Stonepass E2E + health	Full path tests, health score, health panel.	Demo review.	No sharing/public discovery.
7	Phase 5	Prompt-to-World DNA	WorldArchitectAgent, DNA/title/summary/starting beat, preview.	AI key if real generation is requested.	No public UGC.
8	Phase 5	Story graph generation	3-5 beat generated graph, branch support, graph viewer.	Review generated examples.	No marketplace.
9	Phase 6 prep	Persistence/share	DB contracts, save/load, share token.	Supabase setup if DB used.	No public discovery.
10	Phase 6	Fork/remix	Fork, remix, lineage, patch notes.	Review remix controls.	No marketplace.
11	Phase 7	AI Playtester	Path runner, failure detection, report.	Review failed reports.	No auto-approval.
12	Phase 9	Creator Cockpit v1	Inspect/edit panels, health/playtest reports, regenerate, approve/roll back.	Approve editable fields.	No 2D unless 90-day text gate passes.

14. Cursor / Agentic IDE Required Work Loop

1. Confirm source priority, phase, step ID, and why the task belongs in that phase.
2. List allowed scope, including likely files/folders.
3. List blocked scope, especially later-phase work.
4. Make a tiny 3-7 bullet implementation plan.
5. Implement the smallest useful change.
6. Add or update schemas/validators where applicable.

7. Add tests: valid and invalid examples, unit tests, integration tests when crossing systems.
8. Run the narrowest relevant tests first, then broader tests if practical.
9. Report changed files, test command/result, acceptance evidence, validation/fallback, human action, blocked items, next safe task.
10. Stop. Do not proceed to the next step without explicit human approval.

15. Human Operator Guide

Situation	Human operator does	AI tool does
Repo does not exist	Create/confirm GitHub repo and local folder.	Wait or use local-only start only if human confirms.
Package manager unclear	Choose npm unless intentionally changing.	Use only chosen package manager and report version.
AI key needed	Create provider key and add env vars.	Create .env.example/placeholders; never hardcode keys.
Supabase needed	Create project and provide env vars.	Do not invent production DB assumptions before approved step.
Tests fail twice	Review failure and approve next debugging step.	Stop and report likely cause and files touched.
Cursor suggests extra work	Reject or log as future task.	Do not implement extras.
Public sharing requested early	Hold until gates pass.	Refuse implementation and point to blocked gate.

16. Stop Conditions

Stop condition	Required AI response
Credentials are needed	Stop. Ask human owner to add keys through env vars. Provide .env.example if useful.
External account setup is needed	Stop. Explain account/project setup. Do not fake completion.
Task requires later phase work	Stop. Reference blocked phase/gate and offer earliest safe prerequisite.
Tests fail after two focused fixes	Stop. Report failure, likely cause, files touched, and next debugging step.
New dependency seems useful	Stop unless dependency is already approved by stack. Explain why it is needed.
Large refactor is tempting	Stop. Keep current task small and propose refactor separately.
AI output would mutate ledger directly	Stop. Route through validator and deterministic Consequence Engine.

17. Step Cards and Cursor Prompts

Each step below is self-contained. The human operator should copy exactly one step prompt into Cursor, require the pre-coding plan, approve or reject that plan, then require a completion report before moving to the next step.

Week 1

W1-S1 - Create repo and app skeleton

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create playable-worlds-lab workspace with Next.js, TypeScript, Tailwind, ESLint/Prettier, Vitest, and package folders.
Human action before step	Human confirms/create GitHub repo, local folder, npm as package manager unless intentionally changed.
AI allowed scope	repo root, apps/web, packages/core, packages/ai, packages/content, package.json, tsconfig files, lint/test config, README, .nvmrc.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Package/runtime setup lock.
Validator required	None yet beyond project smoke test.
Tests required	Add one smoke test and run npm test or the narrowest available test command.
Done when	Project runs locally, basic home page loads, npm test runs, npm/Node version reported.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S1

- Step name: Create repo and app skeleton

Goal: Create playable-worlds-lab workspace with Next.js, TypeScript, Tailwind, ESLint/Prettier, Vitest, and package folders.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: repo root, apps/web, packages/core, packages/ai, packages/content, package.json, tsconfig files, lint/test config, README, .nvmrc.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Package/runtime setup lock.

Validation required: None yet beyond project smoke test.

Testing required: Add one smoke test and run npm test or the narrowest available test command.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Project runs locally, basic home page loads, npm test runs, npm/Node version reported.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched

- Next safe task only

Stop after this step.

W1-S2 - Create WorldDNA schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create Zod schema and TypeScript type for WorldDNA with safetyMode limited to teen/adult.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, packages/core/tests/unit/schemas, packages/content/examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDNA, SafetyMode.
Validator required	Reject missing fields, unsupported enums, and any safetyMode outside teen/adult.
Tests required	Valid teen/adult examples pass; invalid mode and missing fields fail.
Done when	WorldDNA schema exports and examples validate.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S2
- Step name: Create WorldDNA schema

Goal: Create Zod schema and TypeScript type for WorldDNA with safetyMode limited to teen/adult.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, packages/core/tests/unit/schemas, packages/content/examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDNA, SafetyMode.

Validation required: Reject missing fields, unsupported enums, and any safetyMode outside teen/adult.

Testing required: Valid teen/adult examples pass; invalid mode and missing fields fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: WorldDNA schema exports and examples validate.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S3 - Create PlayerChoice schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Define the choice object used by StoryBeat and runtime choice resolution.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, schema tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	PlayerChoice.
Validator required	Non-empty id/label; optional requiredFlags/blockedByFlags; required consequenceId.
Tests required	Valid choice passes; missing id/label/consequenceId and malformed flags fail.
Done when	Choices validate with deterministic consequence reference.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S3
- Step name: Create PlayerChoice schema

Goal: Define the choice object used by StoryBeat and runtime choice resolution.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human

setup.

5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, schema tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: PlayerChoice.

Validation required: Non-empty id/label; optional requiredFlags/blockedByFlags; required consequenceId.

Testing required: Valid choice passes; missing id/label/consequenceId and malformed flags fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Choices validate with deterministic consequence reference.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S4 - Create StoryBeat schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create StoryBeat schema with trigger, choices, possible consequences, required/blocked flags, and ending/hidden markers.

Field	Instruction
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	StoryBeat + PlayerChoice composition.
Validator required	Require id/title/description/trigger/availableChoices/possibleConsequences.
Tests required	Valid beat passes; no choices, malformed flags, and empty IDs fail.
Done when	StoryBeat validates only when it has a clear playable structure.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S4
- Step name: Create StoryBeat schema

Goal: Create StoryBeat schema with trigger, choices, possible consequences, required/blocked flags, and ending/hidden markers.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery;

no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: StoryBeat + PlayerChoice composition.

Validation required: Require

id/title/description/trigger/availableChoices/possibleConsequences.

Testing required: Valid beat passes; no choices, malformed flags, and empty IDs fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: StoryBeat validates only when it has a clear playable structure.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S5 - Create Consequence schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create Consequence schema for flags, visible changes, goals, NPC updates, and temporary instance references.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no

Field	Instruction
	hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Consequence.
Validator required	Require id/summary; default or require arrays consistently; reject malformed state-change lists.
Tests required	Valid fight-ogre-style consequence passes; missing summary/invalid arrays fail.
Done when	Consequence objects validate and default safely.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S5
- Step name: Create Consequence schema

Goal: Create Consequence schema for flags, visible changes, goals, NPC updates, and temporary instance references.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Consequence.

Validation required: Require id/summary; default or require arrays consistently; reject malformed state-change lists.

Testing required: Valid fight-ogre-style consequence passes; missing summary/invalid

arrays fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Consequence objects validate and default safely.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S6 - Create WorldLedger and WorldEvent schemas

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create WorldLedger schema and WorldEvent sub-schema for flags, events, discovered locations, and unlocked goals.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldLedger, WorldEvent.
Validator required	WorldEvent requires id/type/summary/turnNumber; ledger arrays validate.

Field	Instruction
Tests required	Empty ledger and post-ogre event ledger pass; malformed event fails.
Done when	Ledger starts empty, validates after events, and rejects malformed records.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S6
- Step name: Create WorldLedger and WorldEvent schemas

Goal: Create WorldLedger schema and WorldEvent sub-schema for flags, events, discovered locations, and unlocked goals.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldLedger, WorldEvent.

Validation required: WorldEvent requires id/type/summary/turnNumber; ledger arrays validate.

Testing required: Empty ledger and post-ogre event ledger pass; malformed event fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.

- Do not continue into the next step without explicit human approval.

Done when: Ledger starts empty, validates after events, and rejects malformed records.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S7 - Create DirectorDecision schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create structured AI output schema for DirectorDecision.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	DirectorDecision.
Validator required	Allowed action enum only; targetId and reason required; confidence between 0 and 1.
Tests required	Valid decision passes; invalid action/missing target/out-of-range confidence fail.
Done when	Only allowed DirectorDecision actions pass.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S7
- Step name: Create DirectorDecision schema

Goal: Create structured AI output schema for DirectorDecision.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: DirectorDecision.

Validation required: Allowed action enum only; targetId and reason required; confidence between 0 and 1.

Testing required: Valid decision passes; invalid action/missing target/out-of-range confidence fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Only allowed DirectorDecision actions pass.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required

- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S8 - Create TemporaryInstance schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create schema for short-lived caves, ruins, trials, dreams, and dungeons.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	TemporaryInstance, TemporaryInstanceRoom.
Validator required	Require entry flags, entrance text, rooms, completion condition, completion consequence, cleanup behavior.
Tests required	Valid cave passes; missing entrance/completion/room shape fails.
Done when	Temporary instances validate before Phase 3 uses them.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S8
- Step name: Create TemporaryInstance schema

Goal: Create schema for short-lived caves, ruins, trials, dreams, and dungeons.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: TemporaryInstance, TemporaryInstanceRoom.

Validation required: Require entry flags, entrance text, rooms, completion condition, completion consequence, cleanup behavior.

Testing required: Valid cave passes; missing entrance/completion/room shape fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Temporary instances validate before Phase 3 uses them.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S9 - Create NPC schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create minimal NPC profile and tone-rule

Field	Instruction
Human action before step	schema so NPC shape is not invented later.
AI allowed scope	None unless external setup is discovered.
AI blocked scope	packages/core/schemas, tests, examples, packages/content/examples.
Contracts touched	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Validator required	NPC.
Tests required	Require id/name/role/description; restrict attitude enum; validate toneRules and knownFlags arrays.
Done when	Valid ogre/elder NPC examples pass; missing name and invalid attitude fail.
Next allowed step	NPC profiles validate and export for Stonepass content.
	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S9
- Step name: Create NPC schema

Goal: Create minimal NPC profile and tone-rule schema so NPC shape is not invented later.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, tests, examples, packages/content/examples.
Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: NPC.

Validation required: Require id/name/role/description; restrict attitude enum; validate toneRules and knownFlags arrays.

Testing required: Valid ogre/elder NPC examples pass; missing name and invalid attitude fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: NPC profiles validate and export for Stonepass content.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S10 - Create WorldDefinition schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create top-level complete world contract that composes WorldDNA, StoryBeat, Consequence, NPC, and TemporaryInstance.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no

Field	Instruction
	marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition.
Validator required	Require schemaVersion/id/title/summary/worldDNA/startingBeatId/storyBeats/consequences/npcs.
Tests required	Minimal valid Stonepass-like world passes; missing startingBeatId or components fail.
Done when	Complete world object can validate as one object.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S10
- Step name: Create WorldDefinition schema

Goal: Create top-level complete world contract that composes WorldDNA, StoryBeat, Consequence, NPC, and TemporaryInstance.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition.

Validation required: Require
schemaVersion/id/title/summary/worldDNA/startingBeatId/storyBeats/consequences/
npcs.

Testing required: Minimal valid Stonepass-like world passes; missing startingBeatId or
components fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Complete world object can validate as one object.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S11 - Create WorldSession schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create current play-state contract for current beat, ledger, active instance, turn number, choice history, and debug events.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, packages/core/session, tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no

Field	Instruction
	cross-phase work.
Contracts touched	WorldSession.
Validator required	Require schemaVersion/worldId/worldVersionId/currentBeatId/ledger/turnNumber/ choiceHistory/debugEvents.
Tests required	New session passes; malformed ledger/currentBeatId/turn number fails.
Done when	Session state is canonical and not random React state.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S11
- Step name: Create WorldSession schema

Goal: Create current play-state contract for current beat, ledger, active instance, turn number, choice history, and debug events.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, packages/core/session, tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldSession.

Validation required: Require
schemaVersion/worldId/worldVersionId/currentBeatId/ledger/turnNumber/
choiceHistory/debugEvents.

Testing required: New session passes; malformed ledger/currentBeatId/turn number fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Session state is canonical and not random React state.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S12 - Create DebugEvent schema

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create debug trace contract for choices, consequences, flags, goals, AI suggestions, fallbacks, and validation failures.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas, packages/core/debug, tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	DebugEvent.
Validator required	Require id/turnNumber/type/summary;

Field	Instruction
Tests required	restrict type enum. Valid debug events pass; invalid type/missing summary fail.
Done when	DebugEvent is available for runtime and UI traces.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S12
- Step name: Create DebugEvent schema

Goal: Create debug trace contract for choices, consequences, flags, goals, AI suggestions, fallbacks, and validation failures.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas, packages/core/debug, tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: DebugEvent.

Validation required: Require id/turnNumber/type/summary; restrict type enum.

Testing required: Valid debug events pass; invalid type/missing summary fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed,

stop and tell the human owner exactly what to do.

- Do not continue into the next step without explicit human approval.

Done when: DebugEvent is available for runtime and UI traces.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S13 - Create AIResult contract

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create wrapper for AI output with validation status, raw output, provider, fallback, errors, latency, and seed.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/schemas or packages/ai/contracts, tests, examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIResult.
Validator required	Require ok/provider/fallbackUsed; validationErrors optional; seed optional.
Tests required	Valid success/failure AIResult objects pass; malformed provider/fallback shape fails.
Done when	AI calls have a canonical result wrapper before runtime use.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S13
- Step name: Create AIResult contract

Goal: Create wrapper for AI output with validation status, raw output, provider, fallback, errors, latency, and seed.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/schemas or packages/ai/contracts, tests, examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: AIResult<T>.

Validation required: Require ok/provider/fallbackUsed; validationErrors optional; seed optional.

Testing required: Valid success/failure AIResult objects pass; malformed provider/fallback shape fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: AI calls have a canonical result wrapper before runtime use.

Completion report required:

- Task completed
- Files changed and why

- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S14 - Create cross-file world validator

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create validateWorldDefinition.ts to check references, graph integrity, duplicate IDs, reachable beats, and known flags.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/validators, tests, Stonepass examples.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition plus all referenced schemas.
Validator required	Check startingBeatId, choice consequenceIds, possibleConsequences, temporaryInstances, completionConsequenceId, duplicate IDs, flags, unreachable beats, dead ends.
Tests required	Valid Stonepass-like world passes; missing consequence, duplicate ID, unreachable beat, and bad flag fail.
Done when	Whole-world validation exists before generated worlds are allowed.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation

- Week: 1
- Step ID: W1-S14
- Step name: Create cross-file world validator

Goal: Create `validateWorldDefinition.ts` to check references, graph integrity, duplicate IDs, reachable beats, and known flags.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: `packages/core/validators`, tests, Stonepass examples.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: `WorldDefinition` plus all referenced schemas.

Validation required: Check `startingBeatId`, `choice consequenceIds`, `possibleConsequences`, `temporaryInstances`, `completionConsequenceId`, duplicate IDs, flags, unreachable beats, dead ends.

Testing required: Valid Stonepass-like world passes; missing consequence, duplicate ID, unreachable beat, and bad flag fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Whole-world validation exists before generated worlds are allowed.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added

- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S15 - Create Stonepass Valley world JSON

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create manual Stonepass world data for ogre bridge scenario with fight, trick, feed, talk, and sneak choices.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/content/worlds/stonepass, examples, validation tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, WorldDNA, StoryBeat, PlayerChoice, Consequence, NPC, TemporaryInstance references as needed.
Validator required	Stonepass must pass individual schemas and validateWorldDefinition.
Tests required	Stonepass data validates; invalid fixture fails.
Done when	Stonepass can be represented fully in JSON and validated.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S15
- Step name: Create Stonepass Valley world JSON

Goal: Create manual Stonepass world data for ogre bridge scenario with fight, trick, feed,

talk, and sneak choices.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/content/worlds/stonepass, examples, validation tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, WorldDNA, StoryBeat, PlayerChoice, Consequence, NPC, TemporaryInstance references as needed.

Validation required: Stonepass must pass individual schemas and validateWorldDefinition.

Testing required: Stonepass data validates; invalid fixture fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Stonepass can be represented fully in JSON and validated.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W1-S16 - Create minimal AIProvider interface and FakeProvider

Field	Instruction
Phase	Phase 0 - Foundation
Goal	Create provider-agnostic AI contract and deterministic fake provider for tests, without real API calls.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/ai/gateway, packages/ai/providers, packages/ai/tests, .env.example.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIProvider, AIRequest, FakeProvider, AIResult integration.
Validator required	FakeProvider validates structured output through provided schema and can simulate failure.
Tests required	FakeProvider valid result passes; invalid result and provider failure are testable.
Done when	FakeProvider can return a valid DirectorDecision and failure cases before real AI work.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 0 - Foundation
- Week: 1
- Step ID: W1-S16
- Step name: Create minimal AIProvider interface and FakeProvider

Goal: Create provider-agnostic AI contract and deterministic fake provider for tests, without real API calls.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/gateway, packages/ai/providers, packages/ai/tests, .env.example.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: AIProvider, AIRequest, FakeProvider, AIResult integration.

Validation required: FakeProvider validates structured output through provided schema and can simulate failure.

Testing required: FakeProvider valid result passes; invalid result and provider failure are testable.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: FakeProvider can return a valid DirectorDecision and failure cases before real AI work.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 2

W2-S1 - Build world JSON loader

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Load a validated WorldDefinition from content files.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/world, packages/content/worlds/stonepass, loader tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition.
Validator required	Call schema parse and validateWorldDefinition.
Tests required	Loader accepts Stonepass and rejects broken fixture.
Done when	Runtime can load validated Stonepass without AI.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 2
- Step ID: W2-S1
- Step name: Build world JSON loader

Goal: Load a validated WorldDefinition from content files.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.

4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/world, packages/content/worlds/stonepass, loader tests.
Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition.

Validation required: Call schema parse and validateWorldDefinition.

Testing required: Loader accepts Stonepass and rejects broken fixture.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Runtime can load validated Stonepass without AI.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W2-S2 - Initialize WorldSession object

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Create a new WorldSession from a WorldDefinition startingBeatId and empty ledger/debug state.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/session, runtime tests.

Field	Instruction
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldSession, WorldLedger, DebugEvent.
Validator required	Validate session after creation.
Tests required	New session starts at startingBeatId with turn 0 and empty choice/debug history.
Done when	Session is valid and ready for play.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 2
- Step ID: W2-S2
- Step name: Initialize WorldSession object

Goal: Create a new WorldSession from a WorldDefinition startingBeatId and empty ledger/debug state.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/session, runtime tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldSession, WorldLedger, DebugEvent.

Validation required: Validate session after creation.

Testing required: New session starts at startingBeatId with turn 0 and empty choice/debug history.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Session is valid and ready for play.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W2-S3 - Build story beat selector

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Select current or next valid StoryBeat from session state and ledger flags.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/story, packages/core/runtime, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	StoryBeat, WorldSession, WorldLedger.
Validator required	Respect requiredFlags, blockedByFlags, hidden/ending markers.

Field	Instruction
Tests required	Available beat selected; blocked beat ignored; missing beat fails gracefully.
Done when	Runtime can select a valid current beat.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 2
- Step ID: W2-S3
- Step name: Build story beat selector

Goal: Select current or next valid StoryBeat from session state and ledger flags.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/story, packages/core/runtime, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: StoryBeat, WorldSession, WorldLedger.

Validation required: Respect requiredFlags, blockedByFlags, hidden/ending markers.

Testing required: Available beat selected; blocked beat ignored; missing beat fails gracefully.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Runtime can select a valid current beat.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W2-S4 - Build choice resolver

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Resolve a player choice only if the choice exists and its requirements are satisfied.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/runtime, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	PlayerChoice, StoryBeat, WorldSession.
Validator required	Reject invalid choice, blocked choice, and unmet requirements.
Tests required	fight/feed/sneak/talk/trick resolve; fake choice fails.
Done when	Invalid choices are blocked deterministically.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 2

- Step ID: W2-S4
- Step name: Build choice resolver

Goal: Resolve a player choice only if the choice exists and its requirements are satisfied.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/runtime, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: PlayerChoice, StoryBeat, WorldSession.

Validation required: Reject invalid choice, blocked choice, and unmet requirements.

Testing required: fight/feed/sneak/talk/trick resolve; fake choice fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Invalid choices are blocked deterministically.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W2-S5 - Apply first consequence through runtime

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Apply a selected consequence through deterministic runtime code.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/runtime, consequence tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Consequence, WorldLedger, DebugEvent, WorldSession.
Validator required	Consequence must exist and be applicable; session validates after update.
Tests required	First ogre consequence updates flags, goal, ledger, debug event.
Done when	A player choice creates a real remembered world change.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 2
- Step ID: W2-S5
- Step name: Apply first consequence through runtime

Goal: Apply a selected consequence through deterministic runtime code.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/runtime, consequence tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Consequence, WorldLedger, DebugEvent, WorldSession.

Validation required: Consequence must exist and be applicable; session validates after update.

Testing required: First ogre consequence updates flags, goal, ledger, debug event.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: A player choice creates a real remembered world change.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W2-S6 - Build text play screen

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Create minimal browser text play screen for Stonepass choices.
Human action before step	None unless external setup is discovered.
AI allowed scope	apps/web features/world-play, minimal components, smoke tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no

Field	Instruction
	marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldSession surface only.
Validator required	UI must use runtime functions, not mutate state directly.
Tests required	Smoke test page renders and offers valid choices.
Done when	User can start Stonepass and choose a valid action in browser.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 2
- Step ID: W2-S6
- Step name: Build text play screen

Goal: Create minimal browser text play screen for Stonepass choices.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: apps/web features/world-play, minimal components, smoke tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldSession surface only.

Validation required: UI must use runtime functions, not mutate state directly.

Testing required: Smoke test page renders and offers valid choices.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: User can start Stonepass and choose a valid action in browser.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W2-S7 - Add manual Stonepass path tests

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Add deterministic tests for the initial ogre choice path without AI.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/tests/integration, apps/web smoke tests if available.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, WorldSession, Consequence.
Validator required	Validate world and session before/after action.
Tests required	Player can start Stonepass, select each ogre choice, and record ledger/debug events.

Field	Instruction
Done when	Phase 1 proof path works without AI.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 2
- Step ID: W2-S7
- Step name: Add manual Stonepass path tests

Goal: Add deterministic tests for the initial ogre choice path without AI.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/tests/integration, apps/web smoke tests if available.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, WorldSession, Consequence.

Validation required: Validate world and session before/after action.

Testing required: Player can start Stonepass, select each ogre choice, and record ledger/debug events.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Phase 1 proof path works without AI.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 3

W3-S1 - Build Consequence Engine core function

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Centralize consequence application in a deterministic engine function.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/consequence, packages/core/runtime, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Consequence, WorldLedger, WorldSession.
Validator required	Validate input consequence and post-state.
Tests required	addFlags/removeFlags/unlockGoals/visibleChanges are applied correctly.
Done when	No feature bypasses the Consequence Engine.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 3

- Step ID: W3-S1
- Step name: Build Consequence Engine core function

Goal: Centralize consequence application in a deterministic engine function.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/consequence, packages/core/runtime, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Consequence, WorldLedger, WorldSession.

Validation required: Validate input consequence and post-state.

Testing required: addFlags/removeFlags/unlockGoals/visibleChanges are applied correctly.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: No feature bypasses the Consequence Engine.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W3-S2 - Validate consequence preconditions

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Block consequences when prerequisites or session/world references are invalid.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/consequence, validators, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Consequence, WorldDefinition, WorldSession.
Validator required	Check existence, required flags, blocked flags, known references.
Tests required	Unmet prereq and fake consequence fail deterministically.
Done when	Bad consequence application is rejected.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 3
- Step ID: W3-S2
- Step name: Validate consequence preconditions

Goal: Block consequences when prerequisites or session/world references are invalid.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human

setup.

5. Confirm you will not work outside this step.

Allowed scope: packages/core/consequence, validators, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Consequence, WorldDefinition, WorldSession.

Validation required: Check existence, required flags, blocked flags, known references.

Testing required: Unmet prereq and fake consequence fail deterministically.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Bad consequence application is rejected.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W3-S3 - Finalize flag lifecycle rules

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Define how activeFlags, resolvedFlags, requiredFlags, and blockedByFlags behave.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/ledger, packages/core/consequence, tests, docs.
AI blocked scope	No 2D/3D rendering; no real-time

Field	Instruction
	multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldLedger flag model.
Validator required	No duplicate flags; add/remove/resolve logic consistent.
Tests required	Flag add/remove/resolve paths pass.
Done when	Flags behave predictably across choices and beats.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 3
- Step ID: W3-S3
- Step name: Finalize flag lifecycle rules

Goal: Define how activeFlags, resolvedFlags, requiredFlags, and blockedByFlags behave.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/ledger, packages/core/consequence, tests, docs.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldLedger flag model.

Validation required: No duplicate flags; add/remove/resolve logic consistent.

Testing required: Flag add/remove/resolve paths pass.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Flags behave predictably across choices and beats.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W3-S4 - Build World Ledger UI panel

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Show active flags, resolved flags, events, discovered locations, and unlocked goals in browser/debug UI.
Human action before step	None unless external setup is discovered.
AI allowed scope	apps/web/features/world-debug, components, smoke tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldLedger.
Validator required	Read-only panel; no direct mutation.
Tests required	Panel renders ledger entries after a choice.
Done when	Human can inspect what the world

Field	Instruction
Next allowed step	remembers. Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 3
- Step ID: W3-S4
- Step name: Build World Ledger UI panel

Goal: Show active flags, resolved flags, events, discovered locations, and unlocked goals in browser/debug UI.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: apps/web/features/world-debug, components, smoke tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldLedger.

Validation required: Read-only panel; no direct mutation.

Testing required: Panel renders ledger entries after a choice.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Human can inspect what the world remembers.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W3-S5 - Add debug log model usage

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Use DebugEvent schema for every important runtime transition.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/debug, runtime integration, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	DebugEvent.
Validator required	Every DebugEvent validates; include validation failures and fallback use where applicable.
Tests required	Choice selected, consequence applied, flags changed, next goal selected, validation failure logged.
Done when	Every world change can be traced.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime

- Week: 3
- Step ID: W3-S5
- Step name: Add debug log model usage

Goal: Use DebugEvent schema for every important runtime transition.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/debug, runtime integration, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: DebugEvent.

Validation required: Every DebugEvent validates; include validation failures and fallback use where applicable.

Testing required: Choice selected, consequence applied, flags changed, next goal selected, validation failure logged.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Every world change can be traced.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched

- Next safe task only

Stop after this step.

W3-S6 - Build debug log UI panel

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Render DebugEvent entries for the human/operator.
Human action before step	None unless external setup is discovered.
AI allowed scope	apps/web/features/world-debug, components, smoke tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	DebugEvent.
Validator required	Read-only UI; no mutation.
Tests required	Debug panel displays choice/consequence/flag/goal entries.
Done when	Operator can see why the world changed.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 3
- Step ID: W3-S6
- Step name: Build debug log UI panel

Goal: Render DebugEvent entries for the human/operator.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human

setup.

5. Confirm you will not work outside this step.

Allowed scope: apps/web/features/world-debug, components, smoke tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: DebugEvent.

Validation required: Read-only UI; no mutation.

Testing required: Debug panel displays choice/consequence/flag/goal entries.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Operator can see why the world changed.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W3-S7 - Phase 1 acceptance hardening

Field	Instruction
Phase	Phase 1 - Text Runtime
Goal	Run the full Phase 1 acceptance path and close obvious runtime gaps.
Human action before step	None unless external setup is discovered.
AI allowed scope	narrow runtime tests, small bug fixes within Phase 1 scope.
AI blocked scope	No 2D/3D rendering; no real-time

Field	Instruction
	multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, WorldSession, WorldLedger, DebugEvent.
Validator required	World and session validation must pass after every tested action.
Tests required	Stonepass starts, choices resolve, consequences apply, ledger/debug update, invalid choice blocked.
Done when	Phase 1 gate is ready for human approval.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 1 - Text Runtime
- Week: 3
- Step ID: W3-S7
- Step name: Phase 1 acceptance hardening

Goal: Run the full Phase 1 acceptance path and close obvious runtime gaps.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: narrow runtime tests, small bug fixes within Phase 1 scope.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, WorldSession, WorldLedger, DebugEvent.

Validation required: World and session validation must pass after every tested action.

Testing required: Stonepass starts, choices resolve, consequences apply, ledger/debug update, invalid choice blocked.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Phase 1 gate is ready for human approval.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 4

W4-S1 - Build AI Gateway around provider contracts

Field	Instruction
Phase	Phase 2 - AI Director v1
Goal	Create AI Gateway that routes structured calls through providers and schemas.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/ai/gateway, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIProvider, AIRequest, AIResult.
Validator required	All provider output must validate against

Field	Instruction
	requested schema.
Tests required	Gateway returns AIResult success, validation failure, and fallback states.
Done when	No system calls provider directly.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 2 - AI Director v1
- Week: 4
- Step ID: W4-S1
- Step name: Build AI Gateway around provider contracts

Goal: Create AI Gateway that routes structured calls through providers and schemas.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/gateway, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: AIProvider, AIRequest, AIResult.

Validation required: All provider output must validate against requested schema.

Testing required: Gateway returns AIResult success, validation failure, and fallback states.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.

- Do not continue into the next step without explicit human approval.

Done when: No system calls provider directly.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W4-S2 - Expand FakeProvider scenarios

Field	Instruction
Phase	Phase 2 - AI Director v1
Goal	Add deterministic canned responses, invalid output, thrown failure, and seed support.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/ai/providers/fakeProvider, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	FakeProvider, AIResult.
Validator required	Fake responses validate or fail predictably.
Tests required	Valid, invalid, thrown error, and seeded outputs are covered.
Done when	Local/test AI behavior is reproducible.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 2 - AI Director v1

- Week: 4
- Step ID: W4-S2
- Step name: Expand FakeProvider scenarios

Goal: Add deterministic canned responses, invalid output, thrown failure, and seed support.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/providers/fakeProvider, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: FakeProvider, AIResult.

Validation required: Fake responses validate or fail predictably.

Testing required: Valid, invalid, thrown error, and seeded outputs are covered.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Local/test AI behavior is reproducible.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W4-S3 - Create OpenAIProvider placeholder or implementation

Field	Instruction
Phase	Phase 2 - AI Director v1
Goal	Create safe provider placeholder or env-var-backed implementation without hardcoded secrets.
Human action before step	Human provides API key through env vars only if real provider calls are requested.
AI allowed scope	packages/ai/providers, .env.example, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIProvider, AIResult.
Validator required	Never return raw output directly into gameplay.
Tests required	No-key placeholder path works; env var names documented; no secrets committed.
Done when	Real-provider path is safe and optional.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 2 - AI Director v1
- Week: 4
- Step ID: W4-S3
- Step name: Create OpenAIProvider placeholder or implementation

Goal: Create safe provider placeholder or env-var-backed implementation without hardcoded secrets.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.

3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/providers, .env.example, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: AIProvider, AIResult.

Validation required: Never return raw output directly into gameplay.

Testing required: No-key placeholder path works; env var names documented; no secrets committed.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Real-provider path is safe and optional.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W4-S4 - Build DirectorAgent

Field	Instruction
Phase	Phase 2 - AI Director v1
Goal	Use AI Gateway to suggest next beat, recap, session wrap-up, or instance request without mutating state.

Field	Instruction
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/ai/agents/directorAgent, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	DirectorDecision, AIResult, WorldSession.
Validator required	DirectorDecision schema validation and fallback required.
Tests required	Valid suggestion passes; invalid output rejected; fallback fires.
Done when	AI can suggest but cannot execute permanent truth changes.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 2 - AI Director v1
- Week: 4
- Step ID: W4-S4
- Step name: Build DirectorAgent

Goal: Use AI Gateway to suggest next beat, recap, session wrap-up, or instance request without mutating state.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/agents/directorAgent, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar

system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: DirectorDecision, AIResult, WorldSession.

Validation required: DirectorDecision schema validation and fallback required.

Testing required: Valid suggestion passes; invalid output rejected; fallback fires.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: AI can suggest but cannot execute permanent truth changes.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W4-S5 - Build NPCReactionAgent

Field	Instruction
Phase	Phase 2 - AI Director v1
Goal	Generate short NPC reactions within tone and safety rules.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/ai/agents/npcAgent, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	NPC, AIResult, safetyMode.

Field	Instruction
Validator required	Tone/safety constraints, max length, no direct ledger mutation.
Tests required	Ogre/elder reaction examples pass; unsafe/off-tone output rejected or falls back.
Done when	NPC flavor can adapt safely.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 2 - AI Director v1
- Week: 4
- Step ID: W4-S5
- Step name: Build NPCReactionAgent

Goal: Generate short NPC reactions within tone and safety rules.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/agents/npcAgent, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: NPC, AIResult, safetyMode.

Validation required: Tone/safety constraints, max length, no direct ledger mutation.

Testing required: Ogre/elder reaction examples pass; unsafe/off-tone output rejected or falls back.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed,

stop and tell the human owner exactly what to do.

- Do not continue into the next step without explicit human approval.

Done when: NPC flavor can adapt safely.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W4-S6 - Integrate fallback and failure behavior

Field	Instruction
Phase	Phase 2 - AI Director v1
Goal	Ensure game remains playable when AI fails, times out, or returns invalid data.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/ai/gateway, packages/core/runtime integration, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIResult, DebugEvent.
Validator required	Fallback must log DebugEvent and never block gameplay.
Tests required	AI failure still lets player continue; fallback_used debug event appears.
Done when	Runtime does not depend on successful AI calls.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 2 - AI Director v1
- Week: 4
- Step ID: W4-S6
- Step name: Integrate fallback and failure behavior

Goal: Ensure game remains playable when AI fails, times out, or returns invalid data.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/gateway, packages/core/runtime integration, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: AIResult, DebugEvent.

Validation required: Fallback must log DebugEvent and never block gameplay.

Testing required: AI failure still lets player continue; fallback_used debug event appears.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Runtime does not depend on successful AI calls.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required

- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W4-S7 - Show Director reasoning panel

Field	Instruction
Phase	Phase 2 - AI Director v1
Goal	Display AI suggestions, confidence, reason, fallback state, and validation errors in debug UI.
Human action before step	None unless external setup is discovered.
AI allowed scope	apps/web/features/world-debug, components, smoke tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	DirectorDecision, AIResult, DebugEvent.
Validator required	Read-only panel; no direct state mutation.
Tests required	Panel shows valid suggestion and fallback case.
Done when	Human can inspect AI reasoning safely.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 2 - AI Director v1
- Week: 4
- Step ID: W4-S7
- Step name: Show Director reasoning panel

Goal: Display AI suggestions, confidence, reason, fallback state, and validation errors in debug UI.

Before coding:

1. Read the relevant project files and this step card.

2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: apps/web/features/world-debug, components, smoke tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: DirectorDecision, AIResult, DebugEvent.

Validation required: Read-only panel; no direct state mutation.

Testing required: Panel shows valid suggestion and fallback case.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Human can inspect AI reasoning safely.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 5

W5-S1 - Generate or load cave from cave_exposed flag

Field	Instruction
Phase	Phase 3 - Temporary Instances
Goal	Activate a short cave instance only when

Field	Instruction
Human action before step	Stonepass consequence exposes it.
AI allowed scope	None unless external setup is discovered.
AI blocked scope	packages/core/instances, packages/content/worlds/stonepass, tests.
Contracts touched	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Validator required	TemporaryInstance, WorldSession.
Tests required	cave_exposed required; instance reference must exist.
Done when	No flag blocks cave; cave_exposed allows cave entry.
Next allowed step	Cave cannot appear without correct world state.
	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 3 - Temporary Instances
- Week: 5
- Step ID: W5-S1
- Step name: Generate or load cave from cave_exposed flag

Goal: Activate a short cave instance only when Stonepass consequence exposes it.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/instances, packages/content/worlds/stonepass, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery;

no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: TemporaryInstance, WorldSession.

Validation required: cave_exposed required; instance reference must exist.

Testing required: No flag blocks cave; cave_exposed allows cave entry.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Cave cannot appear without correct world state.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W5-S2 - Create room system

Field	Instruction
Phase	Phase 3 - Temporary Instances
Goal	Track currentTemporaryRoomId and move through instance rooms.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/instances, packages/core/session, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no

Field	Instruction
	cross-phase work.
Contracts touched	TemporaryInstanceRoom, WorldSession.
Validator required	Room IDs must exist in active instance.
Tests required	Enter first room, move valid room, reject missing room.
Done when	Player can navigate simple temporary instance rooms.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 3 - Temporary Instances
- Week: 5
- Step ID: W5-S2
- Step name: Create room system

Goal: Track currentTemporaryRoomId and move through instance rooms.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/instances, packages/core/session, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: TemporaryInstanceRoom, WorldSession.

Validation required: Room IDs must exist in active instance.

Testing required: Enter first room, move valid room, reject missing room.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.

- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Player can navigate simple temporary instance rooms.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W5-S3 - Create encounter system

Field	Instruction
Phase	Phase 3 - Temporary Instances
Goal	Add simple encounter interaction inside temporary instance.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/instances, content, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	TemporaryInstanceRoom, Consequence as needed.
Validator required	Encounter text/choice must be valid and bounded.
Tests required	Encounter interaction records ledger/debug event.
Done when	Cave can contain one meaningful encounter.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 3 - Temporary Instances
- Week: 5
- Step ID: W5-S3
- Step name: Create encounter system

Goal: Add simple encounter interaction inside temporary instance.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/instances, content, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: TemporaryInstanceRoom, Consequence as needed.

Validation required: Encounter text/choice must be valid and bounded.

Testing required: Encounter interaction records ledger/debug event.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Cave can contain one meaningful encounter.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required

- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W5-S4 - Create puzzle system

Field	Instruction
Phase	Phase 3 - Temporary Instances
Goal	Add simple puzzle interaction inside temporary instance.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/instances, content, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	TemporaryInstanceRoom.
Validator required	Puzzle must have clear completion condition.
Tests required	Puzzle can be completed and fails gracefully if invalid.
Done when	Cave can contain one meaningful puzzle.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 3 - Temporary Instances
- Week: 5
- Step ID: W5-S4
- Step name: Create puzzle system

Goal: Add simple puzzle interaction inside temporary instance.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human

setup.

5. Confirm you will not work outside this step.

Allowed scope: packages/core/instances, content, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: TemporaryInstanceRoom.

Validation required: Puzzle must have clear completion condition.

Testing required: Puzzle can be completed and fails gracefully if invalid.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Cave can contain one meaningful puzzle.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W5-S5 - Apply instance completion consequence and cleanup

Field	Instruction
Phase	Phase 3 - Temporary Instances
Goal	Resolve the cave, apply completionConsequenceId, and run vanish/collapse/seal/resolve behavior.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/instances, consequence engine, tests.

Field	Instruction
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	TemporaryInstance, Consequence, WorldSession, DebugEvent.
Validator required	Completion consequence must exist; cleanup behavior must be allowed.
Tests required	Completing cave applies consequence and logs cleanup.
Done when	Cave resolves and returns control to main world.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 3 - Temporary Instances
- Week: 5
- Step ID: W5-S5
- Step name: Apply instance completion consequence and cleanup

Goal: Resolve the cave, apply completionConsequenceId, and run vanish/collapse/seal/resolve behavior.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/instances, consequence engine, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: TemporaryInstance, Consequence, WorldSession, DebugEvent.
Validation required: Completion consequence must exist; cleanup behavior must be allowed.

Testing required: Completing cave applies consequence and logs cleanup.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Cave resolves and returns control to main world.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W5-S6 - Awaken dragon and return to main world

Field	Instruction
Phase	Phase 3 - Temporary Instances
Goal	Complete Stonepass cave path by adding dragon_awake and a new main-world goal.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/content/worlds/stonepass, runtime/instance tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Consequence, WorldLedger, WorldSession.

Field	Instruction
Validator required	dragon_awake flag and unlocked goal must validate.
Tests required	ogre -> landslide -> cave -> dragon path passes.
Done when	Phase 3 core fantasy chain works.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 3 - Temporary Instances
- Week: 5
- Step ID: W5-S6
- Step name: Awaken dragon and return to main world

Goal: Complete Stonepass cave path by adding dragon_awake and a new main-world goal.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/content/worlds/stonepass, runtime/instance tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Consequence, WorldLedger, WorldSession.

Validation required: dragon_awake flag and unlocked goal must validate.

Testing required: ogre -> landslide -> cave -> dragon path passes.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed,

stop and tell the human owner exactly what to do.

- Do not continue into the next step without explicit human approval.

Done when: Phase 3 core fantasy chain works.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W5-S7 - Temporary instance acceptance tests

Field	Instruction
Phase	Phase 3 - Temporary Instances
Goal	Add integration tests for the cave flow and failure paths.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/tests/integration, content fixtures.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	TemporaryInstance, WorldSession, DebugEvent.
Validator required	Validate before/after instance entry/completion.
Tests required	Cave requires flag, has rooms/goal/completion/exit, resolves, dragon awakens.
Done when	Phase 3 gate is ready for human approval.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 3 - Temporary Instances
- Week: 5
- Step ID: W5-S7
- Step name: Temporary instance acceptance tests

Goal: Add integration tests for the cave flow and failure paths.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/tests/integration, content fixtures.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: TemporaryInstance, WorldSession, DebugEvent.

Validation required: Validate before/after instance entry/completion.

Testing required: Cave requires flag, has rooms/goal/completion/exit, resolves, dragon awakens.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Phase 3 gate is ready for human approval.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence

- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 6

W6-S1 - Run full Stonepass end-to-end path tests

Field	Instruction
Phase	Stonepass Hardening + World Health Score v1
Goal	Test ogre -> landslide -> cave -> dragon as one complete player journey.
Human action before step	None unless external setup is discovered.
AI allowed scope	integration tests, narrow runtime fixes.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	All MVP runtime contracts.
Validator required	World/session/debug validation at each major transition.
Tests required	Full path passes; invalid branch and missing reference fail.
Done when	Stonepass works as a complete proof path.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Stonepass Hardening + World Health Score v1
- Week: 6
- Step ID: W6-S1
- Step name: Run full Stonepass end-to-end path tests

Goal: Test ogre -> landslide -> cave -> dragon as one complete player journey.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: integration tests, narrow runtime fixes.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: All MVP runtime contracts.

Validation required: World/session/debug validation at each major transition.

Testing required: Full path passes; invalid branch and missing reference fail.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Stonepass works as a complete proof path.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W6-S2 - Create health score categories

Field	Instruction
Phase	Stonepass Hardening + World Health Score v1

Field	Instruction
Goal	Define deterministic scoring categories for generated and official worlds.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/health, tests, docs.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition.
Validator required	Categories must be deterministic and explainable.
Tests required	Scores include goal clarity, consequence quality, completion path, flag consistency, replayability.
Done when	Health score model is defined.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Stonepass Hardening + World Health Score v1
- Week: 6
- Step ID: W6-S2
- Step name: Create health score categories

Goal: Define deterministic scoring categories for generated and official worlds.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/health, tests, docs.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery;

no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition.

Validation required: Categories must be deterministic and explainable.

Testing required: Scores include goal clarity, consequence quality, completion path, flag consistency, replayability.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Health score model is defined.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W6-S3 - Add deterministic world checks

Field	Instruction
Phase	Stonepass Hardening + World Health Score v1
Goal	Implement non-AI world checks for path, flags, goals, dead ends, and references.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/health, validators, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no

Field	Instruction
	hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition.
Validator required	Reuse validateWorldDefinition where possible.
Tests required	Broken world scores low; valid Stonepass passes.
Done when	Health checks catch structural issues without AI.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Stonepass Hardening + World Health Score v1
- Week: 6
- Step ID: W6-S3
- Step name: Add deterministic world checks

Goal: Implement non-AI world checks for path, flags, goals, dead ends, and references.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/health, validators, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition.

Validation required: Reuse validateWorldDefinition where possible.

Testing required: Broken world scores low; valid Stonepass passes.

Implementation requirements:

- Make the smallest useful change.

- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Health checks catch structural issues without AI.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W6-S4 - Add optional AI critic summary

Field	Instruction
Phase	Stonepass Hardening + World Health Score v1
Goal	Use AI to summarize quality concerns only after deterministic checks run.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/ai/agents/criticAgent, health integration, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIResult, DebugEvent.
Validator required	AI critic cannot override deterministic checks.
Tests required	Critic summary present when provider works; fallback if AI fails.
Done when	AI critic is advisory only.

Field	Instruction
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Stonepass Hardening + World Health Score v1
- Week: 6
- Step ID: W6-S4
- Step name: Add optional AI critic summary

Goal: Use AI to summarize quality concerns only after deterministic checks run.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/agents/criticAgent, health integration, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: AIResult, DebugEvent.

Validation required: AI critic cannot override deterministic checks.

Testing required: Critic summary present when provider works; fallback if AI fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: AI critic is advisory only.

Completion report required:

- Task completed

- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W6-S5 - Calculate and expose overall health score

Field	Instruction
Phase	Stonepass Hardening + World Health Score v1
Goal	Combine health checks into one explainable score and category breakdown.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/health, apps/web debug/creator panels, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, DebugEvent as needed.
Validator required	Score must be deterministic for same input.
Tests required	Stonepass acceptable; no-next-goal world low; invalid consequence fails.
Done when	World Health Score v1 exists and is explainable.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Stonepass Hardening + World Health Score v1
- Week: 6
- Step ID: W6-S5
- Step name: Calculate and expose overall health score

Goal: Combine health checks into one explainable score and category breakdown.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/health, apps/web debug/creator panels, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, DebugEvent as needed.

Validation required: Score must be deterministic for same input.

Testing required: Stonepass acceptable; no-next-goal world low; invalid consequence fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: World Health Score v1 exists and is explainable.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W6-S6 - Show health score in debug UI

Field	Instruction
Phase	Stonepass Hardening + World Health Score v1
Goal	Render health score and failed checks in human-facing debug UI.
Human action before step	None unless external setup is discovered.
AI allowed scope	apps/web/features/world-debug, components, smoke tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Health report object.
Validator required	Read-only UI.
Tests required	Panel renders score and broken checks.
Done when	Operator can see world health before generation/share.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Stonepass Hardening + World Health Score v1
- Week: 6
- Step ID: W6-S6
- Step name: Show health score in debug UI

Goal: Render health score and failed checks in human-facing debug UI.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: apps/web/features/world-debug, components, smoke tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Health report object.

Validation required: Read-only UI.

Testing required: Panel renders score and broken checks.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Operator can see world health before generation/share.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 7

W7-S1 - Build WorldArchitectAgent shell

Field	Instruction
Phase	Phase 5 - Prompt-to-World DNA
Goal	Create agent shell that turns user prompt into structured world draft pieces through AI Gateway.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/ai/agents/worldArchitectAgent, tests.
AI blocked scope	No 2D/3D rendering; no real-time

Field	Instruction
	multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIResult, WorldDefinition draft.
Validator required	No raw save; all output must validate before preview.
Tests required	FakeProvider path returns a valid draft shell; invalid output rejected.
Done when	Prompt generation has a safe shell.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Prompt-to-World DNA
- Week: 7
- Step ID: W7-S1
- Step name: Build WorldArchitectAgent shell

Goal: Create agent shell that turns user prompt into structured world draft pieces through AI Gateway.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/agents/worldArchitectAgent, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: AIResult, WorldDefinition draft.

Validation required: No raw save; all output must validate before preview.

Testing required: FakeProvider path returns a valid draft shell; invalid output rejected.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Prompt generation has a safe shell.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W7-S2 - Generate WorldDNA from prompt

Field	Instruction
Phase	Phase 5 - Prompt-to-World DNA
Goal	Generate teen/adult WorldDNA from user prompt.
Human action before step	None unless external setup is discovered.
AI allowed scope	WorldArchitectAgent, schema tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDNA.
Validator required	safetyMode teen/adult only; no unsupported enum values.
Tests required	Fantasy/cozy/mystery prompts produce valid WorldDNA; invalid mode rejected.

Field	Instruction
Done when	Prompt can produce valid WorldDNA.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Prompt-to-World DNA
- Week: 7
- Step ID: W7-S2
- Step name: Generate WorldDNA from prompt

Goal: Generate teen/adult WorldDNA from user prompt.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: WorldArchitectAgent, schema tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDNA.

Validation required: safetyMode teen/adult only; no unsupported enum values.

Testing required: Fantasy/cozy/mystery prompts produce valid WorldDNA; invalid mode rejected.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Prompt can produce valid WorldDNA.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W7-S3 - Generate title and summary

Field	Instruction
Phase	Phase 5 - Prompt-to-World DNA
Goal	Generate world title and summary from prompt and WorldDNA.
Human action before step	None unless external setup is discovered.
AI allowed scope	WorldArchitectAgent, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition title/summary fields.
Validator required	Non-empty, tone-aligned, safety-compliant.
Tests required	Generated title/summary validate and fit prompt.
Done when	Generated world has clear identity.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Prompt-to-World DNA
- Week: 7
- Step ID: W7-S3
- Step name: Generate title and summary

Goal: Generate world title and summary from prompt and WorldDNA.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: WorldArchitectAgent, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition title/summary fields.

Validation required: Non-empty, tone-aligned, safety-compliant.

Testing required: Generated title/summary validate and fit prompt.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Generated world has clear identity.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W7-S4 - Generate starting story beat

Field	Instruction
Phase	Phase 5 - Prompt-to-World DNA
Goal	Generate a starting beat with at least one valid choice and consequence reference.
Human action before step	None unless external setup is discovered.
AI allowed scope	WorldArchitectAgent, content preview, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	StoryBeat, PlayerChoice, Consequence.
Validator required	Starting beat and first consequence validate; reference integrity checked.
Tests required	Generated starting beat has goal/choice/consequence and no broken refs.
Done when	Generated world has a playable starting point.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Prompt-to-World DNA
- Week: 7
- Step ID: W7-S4
- Step name: Generate starting story beat

Goal: Generate a starting beat with at least one valid choice and consequence reference.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: WorldArchitectAgent, content preview, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: StoryBeat, PlayerChoice, Consequence.

Validation required: Starting beat and first consequence validate; reference integrity checked.

Testing required: Generated starting beat has goal/choice/consequence and no broken refs.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Generated world has a playable starting point.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W7-S5 - Validate generated WorldDefinition

Field	Instruction
Phase	Phase 5 - Prompt-to-World DNA
Goal	Assemble generated draft into WorldDefinition and run schema plus cross-file validation.
Human action before step	None unless external setup is discovered.
AI allowed scope	WorldArchitectAgent, validateWorldDefinition integration, tests.
AI blocked scope	No 2D/3D rendering; no real-time

Field	Instruction
	multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition.
Validator required	Full cross-file validator required before preview/save.
Tests required	Good generated fixture passes; missing consequence/unreachable beat fails.
Done when	Generated world draft cannot bypass validator.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Prompt-to-World DNA
- Week: 7
- Step ID: W7-S5
- Step name: Validate generated WorldDefinition

Goal: Assemble generated draft into WorldDefinition and run schema plus cross-file validation.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: WorldArchitectAgent, validateWorldDefinition integration, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition.

Validation required: Full cross-file validator required before preview/save.

Testing required: Good generated fixture passes; missing consequence/unreachable beat fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Generated world draft cannot bypass validator.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W7-S6 - Show generated world preview

Field	Instruction
Phase	Phase 5 - Prompt-to-World DNA
Goal	Show preview for generated world data without public publishing.
Human action before step	None unless external setup is discovered.
AI allowed scope	apps/web/features/world-create, preview components, smoke tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, health score if available.
Validator required	Preview only; do not save as official template

Field	Instruction
	unless approved.
Tests required	Preview displays generated world and validation result.
Done when	User can inspect a generated world before play/save.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Prompt-to-World DNA
- Week: 7
- Step ID: W7-S6
- Step name: Show generated world preview

Goal: Show preview for generated world data without public publishing.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: apps/web/features/world-create, preview components, smoke tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, health score if available.

Validation required: Preview only; do not save as official template unless approved.

Testing required: Preview displays generated world and validation result.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.

- Do not continue into the next step without explicit human approval.

Done when: User can inspect a generated world before play/save.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 8

W8-S1 - Generate 3-5 story beats

Field	Instruction
Phase	Phase 5 - Story Beat Graph Generator
Goal	Generate a small multi-beat graph for a text world.
Human action before step	None unless external setup is discovered.
AI allowed scope	WorldArchitectAgent, story graph generation tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	StoryBeat, Consequence, WorldDefinition.
Validator required	Each beat must have trigger and valid choices or ending.
Tests required	Generated 3-5 beat graph validates.
Done when	Generated world can be more than one starting beat.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Story Beat Graph Generator
- Week: 8
- Step ID: W8-S1
- Step name: Generate 3-5 story beats

Goal: Generate a small multi-beat graph for a text world.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: WorldArchitectAgent, story graph generation tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: StoryBeat, Consequence, WorldDefinition.

Validation required: Each beat must have trigger and valid choices or ending.

Testing required: Generated 3-5 beat graph validates.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Generated world can be more than one starting beat.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required

- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W8-S2 - Connect beats through conditions and flags

Field	Instruction
Phase	Phase 5 - Story Beat Graph Generator
Goal	Connect story beats using requiredFlags, blockedByFlags, and consequences.
Human action before step	None unless external setup is discovered.
AI allowed scope	story graph generation, validators, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	StoryBeat, Consequence, WorldLedger flags.
Validator required	Every dependency must be produced or marked system/external.
Tests required	Flag-connected path passes; bad dependency fails.
Done when	Generated graph has logical conditional flow.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Story Beat Graph Generator
- Week: 8
- Step ID: W8-S2
- Step name: Connect beats through conditions and flags

Goal: Connect story beats using requiredFlags, blockedByFlags, and consequences.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.

4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: story graph generation, validators, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: StoryBeat, Consequence, WorldLedger flags.

Validation required: Every dependency must be produced or marked system/external.

Testing required: Flag-connected path passes; bad dependency fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Generated graph has logical conditional flow.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W8-S3 - Add branch support

Field	Instruction
Phase	Phase 5 - Story Beat Graph Generator
Goal	Support multiple player routes and consequence-driven branches.
Human action before step	None unless external setup is discovered.
AI allowed scope	graph generation, runtime selector tests.
AI blocked scope	No 2D/3D rendering; no real-time

Field	Instruction
	multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	StoryBeat, PlayerChoice, Consequence.
Validator required	Branches cannot soft-lock and must converge or end intentionally.
Tests required	At least two distinct branches validate and play through.
Done when	Generated worlds can support replay variation.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Story Beat Graph Generator
- Week: 8
- Step ID: W8-S3
- Step name: Add branch support

Goal: Support multiple player routes and consequence-driven branches.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: graph generation, runtime selector tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: StoryBeat, PlayerChoice, Consequence.

Validation required: Branches cannot soft-lock and must converge or end intentionally.

Testing required: At least two distinct branches validate and play through.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Generated worlds can support replay variation.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W8-S4 - Render story graph as list or tree

Field	Instruction
Phase	Phase 5 - Story Beat Graph Generator
Goal	Show generated story graph in creator/debug UI.
Human action before step	None unless external setup is discovered.
AI allowed scope	apps/web/features/world-debug or world-create, components, smoke tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition.
Validator required	Read-only visualization first.
Tests required	Graph displays beats, conditions, choices, consequences.

Field	Instruction
Done when	Human can inspect generated structure.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Story Beat Graph Generator
- Week: 8
- Step ID: W8-S4
- Step name: Render story graph as list or tree

Goal: Show generated story graph in creator/debug UI.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: apps/web/features/world-debug or world-create, components, smoke tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition.

Validation required: Read-only visualization first.

Testing required: Graph displays beats, conditions, choices, consequences.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Human can inspect generated structure.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W8-S5 - Validate no dead end unless ending

Field	Instruction
Phase	Phase 5 - Story Beat Graph Generator
Goal	Detect non-ending beats with no valid next action or next hook.
Human action before step	None unless external setup is discovered.
AI allowed scope	validateWorldDefinition, health checks, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	StoryBeat, WorldDefinition.
Validator required	No dead-end unless isEnding; no unreachable unless isHidden/isEnding.
Tests required	Dead-end fixture fails; ending fixture passes.
Done when	Generated graph quality gate works.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 5 - Story Beat Graph Generator
- Week: 8
- Step ID: W8-S5
- Step name: Validate no dead end unless ending

Goal: Detect non-ending beats with no valid next action or next hook.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: validateWorldDefinition, health checks, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: StoryBeat, WorldDefinition.

Validation required: No dead-end unless isEnding; no unreachable unless isHidden/isEnding.

Testing required: Dead-end fixture fails; ending fixture passes.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Generated graph quality gate works.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 9

W9-S1 - Add Supabase project configuration

Field	Instruction
Phase	Phase 6 Prep - Persistence and Share Links
Goal	Add Supabase configuration only after human setup and env vars exist.
Human action before step	Human creates Supabase project and provides URL/key through env vars before real persistence.
AI allowed scope	env examples, db config placeholders, docs, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Persistence config.
Validator required	No hardcoded secrets; local fallback remains available.
Tests required	No-key path fails gracefully or uses local persistence as approved.
Done when	Persistence setup is configured safely.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 Prep - Persistence and Share Links
- Week: 9
- Step ID: W9-S1
- Step name: Add Supabase project configuration

Goal: Add Supabase configuration only after human setup and env vars exist.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.

4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: env examples, db config placeholders, docs, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Persistence config.

Validation required: No hardcoded secrets; local fallback remains available.

Testing required: No-key path fails gracefully or uses local persistence as approved.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Persistence setup is configured safely.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W9-S2 - Create worlds table contract

Field	Instruction
Phase	Phase 6 Prep - Persistence and Share Links
Goal	Define table contract for world records.
Human action before step	Human confirms Supabase setup if real DB migrations are run.
AI allowed scope	db schema/migrations, types, tests.
AI blocked scope	No 2D/3D rendering; no real-time

Field	Instruction
	multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition storage metadata.
Validator required	schemaVersion required; no unvalidated world save.
Tests required	Migration/type tests or local schema tests pass.
Done when	World record contract exists.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 Prep - Persistence and Share Links
- Week: 9
- Step ID: W9-S2
- Step name: Create worlds table contract

Goal: Define table contract for world records.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: db schema/migrations, types, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition storage metadata.

Validation required: schemaVersion required; no unvalidated world save.

Testing required: Migration/type tests or local schema tests pass.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: World record contract exists.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W9-S3 - Create world_versions table contract

Field	Instruction
Phase	Phase 6 Prep - Persistence and Share Links
Goal	Define versioned world storage contract.
Human action before step	None unless external setup is discovered.
AI allowed scope	db schema/migrations, types, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition versioning.
Validator required	parent/version references valid; schemaVersion stored.
Tests required	Version contract supports original and later remix.
Done when	World versions can be tracked safely.

Field	Instruction
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 Prep - Persistence and Share Links
- Week: 9
- Step ID: W9-S3
- Step name: Create world_versions table contract

Goal: Define versioned world storage contract.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: db schema/migrations, types, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition versioning.

Validation required: parent/version references valid; schemaVersion stored.

Testing required: Version contract supports original and later remix.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: World versions can be tracked safely.

Completion report required:

- Task completed

- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W9-S4 - Create world_sessions table contract

Field	Instruction
Phase	Phase 6 Prep - Persistence and Share Links
Goal	Define saved session storage contract.
Human action before step	None unless external setup is discovered.
AI allowed scope	db schema/migrations, types, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldSession.
Validator required	WorldSession must validate before save/load.
Tests required	Session record contract validates.
Done when	Saved sessions have canonical shape.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 Prep - Persistence and Share Links
- Week: 9
- Step ID: W9-S4
- Step name: Create world_sessions table contract

Goal: Define saved session storage contract.

Before coding:

1. Read the relevant project files and this step card.

2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: db schema/migrations, types, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldSession.

Validation required: WorldSession must validate before save/load.

Testing required: Session record contract validates.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Saved sessions have canonical shape.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W9-S5 - Implement save/load world session

Field	Instruction
Phase	Phase 6 Prep - Persistence and Share Links
Goal	Save and resume WorldSession through approved persistence method.
Human action before step	None unless external setup is discovered.

Field	Instruction
AI allowed scope	persistence adapter, session tests, UI smoke tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldSession, WorldDefinition.
Validator required	Validate before save and after load.
Tests required	Progress survives reload; invalid saved session rejected.
Done when	User can resume saved world.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 Prep - Persistence and Share Links
- Week: 9
- Step ID: W9-S5
- Step name: Implement save/load world session

Goal: Save and resume WorldSession through approved persistence method.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: persistence adapter, session tests, UI smoke tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldSession, WorldDefinition.

Validation required: Validate before save and after load.

Testing required: Progress survives reload; invalid saved session rejected.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: User can resume saved world.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W9-S6 - Create share token and open shared world

Field	Instruction
Phase	Phase 6 Prep - Persistence and Share Links
Goal	Create private/unlisted share link for a world snapshot or fresh-start link.
Human action before step	Human confirms link mode: private, unlisted, or internal-only.
AI allowed scope	share token model, route, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, WorldSession optional snapshot.
Validator required	Invalid token fails gracefully; share does not

Field	Instruction
	imply public discovery.
Tests required	Valid link opens same world; invalid token fails.
Done when	Share links work without public marketplace/discovery.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 Prep - Persistence and Share Links
- Week: 9
- Step ID: W9-S6
- Step name: Create share token and open shared world

Goal: Create private/unlisted share link for a world snapshot or fresh-start link.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: share token model, route, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, WorldSession optional snapshot.

Validation required: Invalid token fails gracefully; share does not imply public discovery.

Testing required: Valid link opens same world; invalid token fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.

- Do not continue into the next step without explicit human approval.

Done when: Share links work without public marketplace/discovery.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 10

W10-S1 - Create fork endpoint

Field	Instruction
Phase	Phase 6 - Fork and Remix
Goal	Create endpoint/action to fork a world without mutating the original.
Human action before step	None unless external setup is discovered.
AI allowed scope	fork service/route, persistence tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, world_versions.
Validator required	Source world must validate before fork.
Tests required	Fork creates new world/version and original unchanged.
Done when	Forking is safe.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 - Fork and Remix
- Week: 10
- Step ID: W10-S1
- Step name: Create fork endpoint

Goal: Create endpoint/action to fork a world without mutating the original.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: fork service/route, persistence tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, world_versions.

Validation required: Source world must validate before fork.

Testing required: Fork creates new world/version and original unchanged.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Forking is safe.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required

- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W10-S2 - Copy WorldDefinition and selected session state safely

Field	Instruction
Phase	Phase 6 - Fork and Remix
Goal	Copy world data and optional snapshot state according to selected fork mode.
Human action before step	None unless external setup is discovered.
AI allowed scope	fork service, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, WorldSession.
Validator required	Copied objects validate and receive new IDs/version refs.
Tests required	Snapshot and fresh-start modes behave correctly.
Done when	Fork copies only intended state.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 - Fork and Remix
- Week: 10
- Step ID: W10-S2
- Step name: Copy WorldDefinition and selected session state safely

Goal: Copy world data and optional snapshot state according to selected fork mode.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.

4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: fork service, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, WorldSession.

Validation required: Copied objects validate and receive new IDs/version refs.

Testing required: Snapshot and fresh-start modes behave correctly.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Fork copies only intended state.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W10-S3 - Track parentWorldId and parentVersionId

Field	Instruction
Phase	Phase 6 - Fork and Remix
Goal	Track lineage for forks and remixes.
Human action before step	None unless external setup is discovered.
AI allowed scope	version model, persistence contract, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no

Field	Instruction
	marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	World version metadata.
Validator required	Parent references required for fork/remix descendants.
Tests required	Lineage appears in stored version metadata.
Done when	Fork/remix ancestry is auditable.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 - Fork and Remix
- Week: 10
- Step ID: W10-S3
- Step name: Track parentWorldId and parentVersionId

Goal: Track lineage for forks and remixes.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: version model, persistence contract, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: World version metadata.

Validation required: Parent references required for fork/remix descendants.

Testing required: Lineage appears in stored version metadata.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Fork/remix ancestry is auditable.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W10-S4 - Add remix prompt

Field	Instruction
Phase	Phase 6 - Fork and Remix
Goal	Allow controlled remix instructions for tone/session/consequence intensity and similar safe fields.
Human action before step	None unless external setup is discovered.
AI allowed scope	world-create/remix UI, agent shell, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDNA, WorldDefinition.
Validator required	Remix prompt cannot bypass safetyMode or validators.
Tests required	Allowed remix field changes pass; blocked changes rejected.
Done when	Remix input is controlled.

Field	Instruction
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 - Fork and Remix
- Week: 10
- Step ID: W10-S4
- Step name: Add remix prompt

Goal: Allow controlled remix instructions for tone/session/consequence intensity and similar safe fields.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: world-create/remix UI, agent shell, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDNA, WorldDefinition.

Validation required: Remix prompt cannot bypass safetyMode or validators.

Testing required: Allowed remix field changes pass; blocked changes rejected.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Remix input is controlled.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W10-S5 - Generate remix version

Field	Instruction
Phase	Phase 6 - Fork and Remix
Goal	Create a new valid WorldDefinition version from remix prompt.
Human action before step	None unless external setup is discovered.
AI allowed scope	WorldArchitectAgent remix mode, validators, persistence tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, AIResult.
Validator required	Full schema and cross-file validation before save.
Tests required	Remix creates valid version; invalid remix rejected.
Done when	Remix does not overwrite original.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 - Fork and Remix
- Week: 10
- Step ID: W10-S5
- Step name: Generate remix version

Goal: Create a new valid WorldDefinition version from remix prompt.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: WorldArchitectAgent remix mode, validators, persistence tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, AIResult.

Validation required: Full schema and cross-file validation before save.

Testing required: Remix creates valid version; invalid remix rejected.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Remix does not overwrite original.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W10-S6 - Generate patch notes

Field	Instruction
Phase	Phase 6 - Fork and Remix
Goal	Generate or compute readable patch notes for forks/remixes.
Human action before step	None unless external setup is discovered.
AI allowed scope	version service, AI optional, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition diff, AIResult optional.
Validator required	Patch notes advisory; source data remains truth.
Tests required	Patch notes show changed tone/beats/consequences.
Done when	User can understand what changed.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 6 - Fork and Remix
- Week: 10
- Step ID: W10-S6
- Step name: Generate patch notes

Goal: Generate or compute readable patch notes for forks/remixes.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: version service, AI optional, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition diff, AIResult optional.

Validation required: Patch notes advisory; source data remains truth.

Testing required: Patch notes show changed tone/beats/consequences.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: User can understand what changed.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 11

W11-S1 - Build PlaytesterAgent shell

Field	Instruction
Phase	Phase 7 - AI Playtester v1
Goal	Create playtester agent wrapper and report contract without auto-approving worlds.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/ai/agents/playtesterAgent, core/playtest, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no

Field	Instruction
	social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIResult, playtest report.
Validator required	AI report is advisory; deterministic checks run first.
Tests required	Agent shell returns report or fallback.
Done when	Playtester has a safe shell.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 7 - AI Playtester v1
- Week: 11
- Step ID: W11-S1
- Step name: Build PlaytesterAgent shell

Goal: Create playtester agent wrapper and report contract without auto-approving worlds.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/ai/agents/playtesterAgent, core/playtest, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: AIResult, playtest report.

Validation required: AI report is advisory; deterministic checks run first.

Testing required: Agent shell returns report or fallback.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Playtester has a safe shell.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W11-S2 - Build deterministic path runner

Field	Instruction
Phase	Phase 7 - AI Playtester v1
Goal	Run all reachable story paths through runtime without AI.
Human action before step	None unless external setup is discovered.
AI allowed scope	packages/core/playtest, runtime tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, WorldSession.
Validator required	World/session validates at each simulated step.
Tests required	Stonepass paths execute; broken path fails.
Done when	Worlds can be tested mechanically.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 7 - AI Playtester v1
- Week: 11
- Step ID: W11-S2
- Step name: Build deterministic path runner

Goal: Run all reachable story paths through runtime without AI.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: packages/core/playtest, runtime tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, WorldSession.

Validation required: World/session validates at each simulated step.

Testing required: Stonepass paths execute; broken path fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Worlds can be tested mechanically.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added

- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W11-S3 - Test all story beat paths

Field	Instruction
Phase	Phase 7 - AI Playtester v1
Goal	Enumerate and test all beat/choice paths within sane limits.
Human action before step	None unless external setup is discovered.
AI allowed scope	playtest runner, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	StoryBeat, PlayerChoice, Consequence.
Validator required	No unreachable/dead-end path unless allowed.
Tests required	Multi-branch fixture covered; unreachable fixture flagged.
Done when	Branch coverage is measurable.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 7 - AI Playtester v1
- Week: 11
- Step ID: W11-S3
- Step name: Test all story beat paths

Goal: Enumerate and test all beat/choice paths within sane limits.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.

3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: playtest runner, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: StoryBeat, PlayerChoice, Consequence.

Validation required: No unreachable/dead-end path unless allowed.

Testing required: Multi-branch fixture covered; unreachable fixture flagged.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Branch coverage is measurable.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W11-S4 - Detect missing next goal

Field	Instruction
Phase	Phase 7 - AI Playtester v1
Goal	Flag worlds that leave player without goal or next hook.
Human action before step	None unless external setup is discovered.
AI allowed scope	playtest checks, health checks, tests.

Field	Instruction
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldLedger, Consequence, StoryBeat.
Validator required	Non-ending path must produce next goal/hook.
Tests required	Missing-goal fixture fails; Stonepass passes.
Done when	Worlds cannot silently stall.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 7 - AI Playtester v1
- Week: 11
- Step ID: W11-S4
- Step name: Detect missing next goal

Goal: Flag worlds that leave player without goal or next hook.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: playtest checks, health checks, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldLedger, Consequence, StoryBeat.

Validation required: Non-ending path must produce next goal/hook.

Testing required: Missing-goal fixture fails; Stonepass passes.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Worlds cannot silently stall.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W11-S5 - Detect missing reward or consequence

Field	Instruction
Phase	Phase 7 - AI Playtester v1
Goal	Flag choices that do not produce meaningful consequence or feedback.
Human action before step	None unless external setup is discovered.
AI allowed scope	playtest checks, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	PlayerChoice, Consequence, WorldLedger.
Validator required	Each meaningful choice must produce state/visible change, goal, NPC update, or ending.
Tests required	No-op choice fixture fails; valid consequence passes.

Field	Instruction
Done when	No empty quests or meaningless choices.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 7 - AI Playtester v1
- Week: 11
- Step ID: W11-S5
- Step name: Detect missing reward or consequence

Goal: Flag choices that do not produce meaningful consequence or feedback.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: playtest checks, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: PlayerChoice, Consequence, WorldLedger.

Validation required: Each meaningful choice must produce state/visible change, goal, NPC update, or ending.

Testing required: No-op choice fixture fails; valid consequence passes.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: No empty quests or meaningless choices.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W11-S6 - Detect invalid flag dependency

Field	Instruction
Phase	Phase 7 - AI Playtester v1
Goal	Find required/blocked flags that are never produced or are misspelled.
Human action before step	None unless external setup is discovered.
AI allowed scope	playtest checks, validateWorldDefinition, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	StoryBeat, PlayerChoice, Consequence.
Validator required	Unknown flags fail unless marked system/external.
Tests required	Bad dependency fixture fails.
Done when	Flag logic is protected.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 7 - AI Playtester v1
- Week: 11
- Step ID: W11-S6
- Step name: Detect invalid flag dependency

Goal: Find required/blocked flags that are never produced or are misspelled.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: playtest checks, validateWorldDefinition, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: StoryBeat, PlayerChoice, Consequence.

Validation required: Unknown flags fail unless marked system/external.

Testing required: Bad dependency fixture fails.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Flag logic is protected.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W11-S7 - Generate and store playtest report

Field	Instruction
Phase	Phase 7 - AI Playtester v1
Goal	Create readable stored report with deterministic failures and optional AI critic notes.
Human action before step	None unless external setup is discovered.
AI allowed scope	playtest report model, persistence if available, UI panel, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Playtest report, AIResult optional.
Validator required	Report must separate deterministic errors from AI commentary.
Tests required	Broken world report stored/displayed; valid Stonepass passes.
Done when	Quality gate exists before broader sharing.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 7 - AI Playtester v1
- Week: 11
- Step ID: W11-S7
- Step name: Generate and store playtest report

Goal: Create readable stored report with deterministic failures and optional AI critic notes.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.

5. Confirm you will not work outside this step.

Allowed scope: playtest report model, persistence if available, UI panel, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Playtest report, AIResult optional.

Validation required: Report must separate deterministic errors from AI commentary.

Testing required: Broken world report stored/displayed; valid Stonepass passes.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Quality gate exists before broader sharing.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

Week 12

W12-S1 - Build World DNA viewer/editor

Field	Instruction
Phase	Phase 9 - Creator Cockpit v1
Goal	Show and safely edit allowed WorldDNA fields.
Human action before step	None unless external setup is discovered.
AI allowed scope	apps/web/features/creator or world-debug, components, tests.

Field	Instruction
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDNA, WorldDefinition.
Validator required	Edits validate and cannot add unsupported safetyMode.
Tests required	Allowed edit saves; invalid enum rejected.
Done when	Creator can inspect/edit World DNA safely.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 9 - Creator Cockpit v1
- Week: 12
- Step ID: W12-S1
- Step name: Build World DNA viewer/editor

Goal: Show and safely edit allowed WorldDNA fields.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: apps/web/features/creator or world-debug, components, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDNA, WorldDefinition.

Validation required: Edits validate and cannot add unsupported safetyMode.

Testing required: Allowed edit saves; invalid enum rejected.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Creator can inspect/edit World DNA safely.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W12-S2 - Build Ledger viewer

Field	Instruction
Phase	Phase 9 - Creator Cockpit v1
Goal	Show world memory and session history for creator/debug review.
Human action before step	None unless external setup is discovered.
AI allowed scope	creator/debug UI, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldLedger, WorldSession.
Validator required	Read-only unless explicit approved action exists.
Tests required	Ledger viewer renders events/flags/goals.
Done when	Creator can inspect what happened.

Field	Instruction
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 9 - Creator Cockpit v1
- Week: 12
- Step ID: W12-S2
- Step name: Build Ledger viewer

Goal: Show world memory and session history for creator/debug review.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: creator/debug UI, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldLedger, WorldSession.

Validation required: Read-only unless explicit approved action exists.

Testing required: Ledger viewer renders events/flags/goals.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Creator can inspect what happened.

Completion report required:

- Task completed

- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W12-S3 - Build Story Beat list/tree viewer

Field	Instruction
Phase	Phase 9 - Creator Cockpit v1
Goal	Show story graph structure and validation status.
Human action before step	None unless external setup is discovered.
AI allowed scope	creator/debug UI, graph components, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, StoryBeat.
Validator required	Read-only first; no unsafe graph mutation.
Tests required	Viewer shows beats, choices, conditions, consequences.
Done when	Creator can inspect world structure.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 9 - Creator Cockpit v1
- Week: 12
- Step ID: W12-S3
- Step name: Build Story Beat list/tree viewer

Goal: Show story graph structure and validation status.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: creator/debug UI, graph components, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, StoryBeat.

Validation required: Read-only first; no unsafe graph mutation.

Testing required: Viewer shows beats, choices, conditions, consequences.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Creator can inspect world structure.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W12-S4 - Build AI reasoning panel

Field	Instruction
Phase	Phase 9 - Creator Cockpit v1
Goal	Show Director decisions, fallback states,

Field	Instruction
Human action before step	provider info, validation errors, and latency.
AI allowed scope	None unless external setup is discovered.
AI blocked scope	creator/debug UI, tests.
	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIResult, DirectorDecision, DebugEvent.
Validator required	Read-only; no direct execution button unless gated.
Tests required	Panel displays suggestion and fallback cases.
Done when	Creator can inspect AI behavior.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 9 - Creator Cockpit v1
- Week: 12
- Step ID: W12-S4
- Step name: Build AI reasoning panel

Goal: Show Director decisions, fallback states, provider info, validation errors, and latency.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: creator/debug UI, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no

cross-phase work.

Contracts touched: AIResult, DirectorDecision, DebugEvent.

Validation required: Read-only; no direct execution button unless gated.

Testing required: Panel displays suggestion and fallback cases.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Creator can inspect AI behavior.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W12-S5 - Build Health Score and Playtest Report panels

Field	Instruction
Phase	Phase 9 - Creator Cockpit v1
Goal	Show health score, failed checks, and playtest report.
Human action before step	None unless external setup is discovered.
AI allowed scope	creator UI, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	Health report, playtest report.
Validator required	Deterministic errors are visually distinct

Field	Instruction
Tests required	from AI commentary.
Done when	Valid/broken world reports display.
Next allowed step	Creator sees quality state clearly.
	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 9 - Creator Cockpit v1
- Week: 12
- Step ID: W12-S5
- Step name: Build Health Score and Playtest Report panels

Goal: Show health score, failed checks, and playtest report.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: creator UI, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: Health report, playtest report.

Validation required: Deterministic errors are visually distinct from AI commentary.

Testing required: Valid/broken world reports display.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Creator sees quality state clearly.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W12-S6 - Allow regenerate suggestion action

Field	Instruction
Phase	Phase 9 - Creator Cockpit v1
Goal	Let creator request a new suggestion without auto-saving it.
Human action before step	None unless external setup is discovered.
AI allowed scope	creator UI, agent action, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	AIResult, WorldDefinition draft.
Validator required	Regenerated suggestion must preview and validate before approval.
Tests required	Regenerate returns preview; invalid suggestion rejected.
Done when	Creator can ask AI for help without losing control.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 9 - Creator Cockpit v1

- Week: 12
- Step ID: W12-S6
- Step name: Allow regenerate suggestion action

Goal: Let creator request a new suggestion without auto-saving it.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.
5. Confirm you will not work outside this step.

Allowed scope: creator UI, agent action, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: AIResult, WorldDefinition draft.

Validation required: Regenerated suggestion must preview and validate before approval.

Testing required: Regenerate returns preview; invalid suggestion rejected.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Creator can ask AI for help without losing control.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

W12-S7 - Add approve/rollback flow

Field	Instruction
Phase	Phase 9 - Creator Cockpit v1
Goal	Allow approved creator changes to save and rejected changes to rollback safely.
Human action before step	None unless external setup is discovered.
AI allowed scope	creator UI, version service, tests.
AI blocked scope	No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.
Contracts touched	WorldDefinition, version metadata.
Validator required	Approve only validated objects; rollback restores previous valid version.
Tests required	Approve saves new version; rollback restores old version.
Done when	Creator cockpit can control changes safely.
Next allowed step	Only the next listed step after human approval.

Cursor kickoff prompt:

You are working in Cursor on Playable Worlds Lab.

Current assignment:

- Phase: Phase 9 - Creator Cockpit v1
- Week: 12
- Step ID: W12-S7
- Step name: Add approve/rollback flow

Goal: Allow approved creator changes to save and rejected changes to rollback safely.

Before coding:

1. Read the relevant project files and this step card.
2. State a 3-7 bullet implementation plan.
3. List exact files you expect to create or edit.
4. Confirm source priority, allowed scope, blocked scope, validators, tests, and human setup.

5. Confirm you will not work outside this step.

Allowed scope: creator UI, version service, tests.

Blocked scope: No 2D/3D rendering; no real-time multiplayer; no public UGC discovery; no marketplace; no creator monetization; no social chat or voice; no complex avatar system; no full economy; no PvP; no VR; no hardcoded secrets; no broad refactors; no cross-phase work.

Contracts touched: WorldDefinition, version metadata.

Validation required: Approve only validated objects; rollback restores previous valid version.

Testing required: Approve saves new version; rollback restores old version.

Implementation requirements:

- Make the smallest useful change.
- Use schemas and deterministic validators before runtime depends on data.
- AI may propose or explain, but deterministic engine code owns permanent world state.
- If credentials, accounts, Supabase setup, browser action, billing, or API keys are needed, stop and tell the human owner exactly what to do.
- Do not continue into the next step without explicit human approval.

Done when: Creator cockpit can control changes safely.

Completion report required:

- Task completed
- Files changed and why
- Tests run and exact result
- Acceptance criteria evidence
- Validation/fallback added
- Human action required
- Blocked items intentionally not touched
- Next safe task only

Stop after this step.

18. Definitions of Done and Milestone Gates

30-day gate

Requirement	Evidence
Stonepass playable as text	Browser/manual smoke test and integration test.
Choices apply consequences	Unit/integration tests for fight/feed/sneak/talk/trick.
Ledger records events	Ledger assertions after player action.

Requirement	Evidence
DebugEvent log exists	Debug events for choice, consequence, flags, goals, validation/fallback.
WorldDefinition validates	Stonepass passes schema and cross-file validator.
WorldSession exists	New session and post-choice session validate.
AI Director suggests only	AI cannot mutate ledger; invalid output rejected; fallback fires.
No prompt-to-world started early	Audit step completion records.

90-day gate

Requirement	Evidence
Stonepass cave -> dragon path complete	Full integration test.
Generated worlds validate	Schema + cross-file validator.
World Health Score exists	Broken-world and Stonepass scoring tests.
AI Playtester flags broken worlds	Stored report test.
Share/fork/remix safe	Persistence/version/fork tests.
Creator Cockpit v1 exists	UI smoke tests for inspect/edit/reports.
2D not started before gate	Audit.

19. Do-Not-Build-Yet Register

Blocked until	Do not build
90-day text gate	2D runtime, multiplayer, public discovery, marketplace, avatar cosmetics.
2D runtime works	Co-op, real-time rooms, portal hub, advanced profiles.
Safety and quality gates	Public UGC, public remix libraries, creator monetization, social chat, voice.
Much later explicit approval	3D, VR, real-money economy, large MMO scale, PvP, open scripting.

20. Templates

Feature implementation template

Field	Required answer before coding
Feature name	
Phase	
Purpose	
Allowed scope	
Blocked scope	

Field	Required answer before coding
Inputs	
Outputs	
Schema	
Validator	
Fallback	
UI/debug	
Tests	
Acceptance criteria	
Human setup required	
Stop when	

Completion report template

Cursor completion report

Task completed: <task name>

Phase: <phase number/title>

Files changed:

- <file>: <why changed>

Tests run:

- <command>: <pass/fail>

Acceptance criteria met:

- <criterion>: yes/no/evidence

Validation/fallback added:

- <schema/validator/fallback summary>

Human action required:

- <none or exact required setup>

Blocked items not touched:

- <later-phase items intentionally not implemented>

Next safe task:

- <one next task only, from the roadmap>

Anti-scope-creep checklist

Question	Required answer before continuing
Did this task touch more than one roadmap phase?	If yes, stop and split the work.
Did this task add a new dependency?	If yes, justify or ask human owner first.
Did this task add UI before runtime/schema support exists?	If yes, stop and build foundation first.
Did this task add AI before deterministic fallback exists?	If yes, stop and add fallback.
Did this task let AI mutate ledger/world	If yes, reject and reroute through

Question	Required answer before continuing
truth?	Consequence Engine.
Did this task skip tests?	If yes, it is not complete.
Did this task start 2D before the 90-day text gate?	If yes, revert or pause.
Did this task add public sharing/discovery before gates?	If yes, revert or pause.
Did this task solve an unrequested problem?	If yes, remove or propose separately.

21. Decision Log

Date	Decision	Reason	Affects	Revisit
2026-05-26	Create unified v4.1 document	Avoid cross-referencing three PDFs and patch conflicts.	All implementation work.	Only through new source-of-truth revision.
2026-05-26	Teen/adult only	Remove unsupported lower-age mode and simplify safety contract.	Schemas, prompts, safety rules.	Only via explicit new revision.
2026-05-26	WorldDefinition required	Stop loose JSON files from pretending to be valid worlds.	Phases 0-6.	No.
2026-05-26	Cave proof path before prompt-to-world	Prove manual runtime before asking AI to generate worlds.	Phases 3-5.	No.
2026-05-26	Replay variation as structured layer	Support text-game market patterns without chaotic randomness.	Phase 1+ design.	After Stonepass core works.